

EL LENGUAJE UNIFICADO DE MODELADO. MANUAL DE REFERENCIA

JAMES RUMBAUGH
IVAR JACOBSON
GRADY BOOCH



*La referencia
definitiva de UML
escrita por sus
creadores*



Addison
Wesley

Incluye CD-ROM
con la versión 1.3 de UML





**EL LENGUAJE UNIFICADO
DE MODELADO
MANUAL DE REFERENCIA**

EL LENGUAJE UNIFICADO DE MODELADO MANUAL DE REFERENCIA



James RUMBAUGH
Ivar JACOBSON
Grady BOOCH

RATIONAL SOFTWARE CORPORATION

Traducción:

Salvador Sánchez

Universidad Pontificia de Salamanca en Madrid

Oscar San Juan

Universidad Pontificia de Salamanca en Madrid

Rafael García-Bermejo

Universidad de Salamanca

Revisión Técnica:

Luis Joyanes

Director del Departamento de Lenguajes y Sistemas Informáticos

Universidad Pontificia de Salamanca en Madrid

Carmelo R. Fernández

Director de Sistemas de Información

Cámara de Comercio, Industria y Navegación de Gipuzkoa

Universidad del País Vasco / Euskal Herriko Unibersitatea

Colaboradores en la revisión:

Juan Manuel Cueva

Departamento de Lenguajes y Sistemas Informáticos

Universidad de Oviedo

Miren Bermejo

Departamento de Lenguajes y Sistemas Informáticos

Universidad del País Vasco / Euskal Herriko Unibersitatea

ADDISON WESLEY

Madrid • México • Santafé de Bogotá • Buenos Aires • Caracas • Lima • Montevideo • San Juan
San José • Santiago • São Paulo • Reading, Massachusetts • Harlow, England

J. Rumbaugh, I. Jacobson, G. Booch

EL LENGUAJE UNIFICADO DE MODELADO.

MANUAL DE REFERENCIA

PEARSON EDUCACIÓN, S. A., Madrid, 2000

ISBN: 84-7829-037-0

Materia: Informática 681.3

Formato 195 × 250

Páginas: 552

J. Rumbaugh, I. Jacobson, G. Booch

EL LENGUAJE UNIFICADO DE MODELADO. MANUAL DE REFERENCIA

No está permitida la reproducción total o parcial de esta obra
ni su tratamiento o transmisión por cualquier medio o método
sin autorización escrita de la Editorial.

DERECHOS RESERVADOS

© 2000 respecto a la primera edición en español por:

PEARSON EDUCACIÓN, S. A.

Núñez de Balboa, 120

28006 Madrid

ISBN: 84-7829-037-0

Depósito Legal: TO-1.310-2000

ADDISON WESLEY es un sello editorial autorizado de PEARSON EDUCACIÓN, S. A.

Traducido de:

THE UNIFIED MODELING LANGUAGE. REFERENCE MANUAL

Addison Wesley Longman Inc.

© 1999

ISBN: 0-201-30998-X

Edición en español:

Editor: Andrés Otero

Asistente editorial: Ana Isabel García

Diseño de cubierta: DIGRAF, S. A.

Composición: COPIBOOK, S. L.

Impreso por: GRAFILLES, S.L.

IMPRESO EN ESPAÑA - PRINTED IN SPAIN

Para Madeline, Nick y Alex

—*Jim*



Prefacio	XI
Objetivos	XI
Esquema general del libro	XII
Convenciones en el formato de los artículos de la enciclopedia	XII
Convenciones de sintaxis	XIV
CD	XIV
Información adicional	XV
Reconocimientos	XV
Prólogo a la edición en español	XVII
Nota sobre la traducción	XXI

Parte 1: Antecedentes

Capítulo 1: Perspectiva general de UML	3
Breve resumen de UML	3
Historia de UML	4
Objetivos de UML	7
Áreas conceptuales de UML	8
Sintaxis de los diagramas y las expresiones	10
Capítulo 2: La naturaleza y propósito de los modelos	11
¿Qué es un modelo?	11
¿Para qué sirven los modelos?	11
Niveles de los modelos	13
¿Qué hay en un modelo?	15
¿Cuál es el significado de un modelo?	16

Parte 2: Conceptos de UML

Capítulo 3: Un paseo por UML	21
Vistas de UML	21

Vista estática	22
Vista de los casos de uso	24
Vista de interacción	25
Vista de la máquina de estados	27
Vista de actividades	29
Vistas físicas	29
Vista de gestión del modelo	32
Construcciones de extensión	33
Conexiones entre vistas	34
 Capítulo 4: La vista estática	 37
Descripción	37
Clasificadores	38
Relaciones	41
Asociaciones	42
Generalización	45
Realización	48
Dependencias	50
Restricción	52
Instancias	53
 Capítulo 5: La vista de casos de usos	 55
Descripción	55
Actor	56
Caso de uso	56
 Capítulo 6: La vista de la máquina de estados	 59
Descripción	59
Máquina de estados	59
Evento	60
Estado	62
Transición	62
Estados compuestos	66
 Capítulo 7: La vista de actividades	 71
Descripción	71
Diagrama de actividades	71
Actividades y otras vistas	74
 Capítulo 8: La vista de interacción	 75
Descripción	75
Colaboración	75
Interacción	76
Diagrama de secuencia	76

Activación	77
Diagrama de colaboración	78
Patrones	80
Capítulo 9: Vistas físicas	83
Descripción	83
Componente	83
Nodo	84
Capítulo 10: La vista de gestión del modelo	87
Descripción	87
Paquete	87
Dependencias en los paquetes	88
Dependencia de acceso e importación	89
Modelo y subsistema	89
Capítulo 11: Mecanismos de extensión	91
Descripción	91
Restricción	91
Valor etiquetado	92
Estereotipos	93
Adaptación de UML	94
Capítulo 12: El entorno de UML	95
Descripción	95
Responsabilidades semánticas	95
Responsabilidades de notación	96
Responsabilidades del lenguaje de programación	97
Modelado con herramientas	98
Parte 3: Referencia	
Capítulo 13: Enciclopedia de términos	103
Capítulo 14: Elementos estándar	479
Parte 4: Apéndices	
Apéndice A: Metamodelo de UML	495
Documentos de definición de UML	495
Estructura del metamodelo	495

Paquete de fundamentos	496
Paquete de elementos de comportamiento	497
Paquete de administración de modelos	497
 Apéndice B: Resumen de la notación	 499
 Apéndice C: Extensiones de proceso	 511
Cómo personalizar UML	511
Extensiones del proceso de desarrollo del software	511
Extensiones de modelado de negocios	513
 Bibliografía	 517
 Índice	 519



Objetivos

Este libro está destinado a ser una referencia útil y completa sobre el Lenguaje Unificado de Modelado para el desarrollador, arquitecto, gestor de proyecto, ingeniero de sistemas, programador, analista, contratista, cliente, y cualquier otro que necesite especificar, diseñar, construir o entender sistemas de software complejos. Proporciona una referencia completa sobre los conceptos y construcciones de UML, incluyendo su semántica, sintaxis, notación y propósito. Está organizado para ser una útil, pero minuciosa, referencia para el desarrollador profesional. También intenta ofrecer detalles adicionales sobre temas que no están claros en la documentación estándar y ofrece una razón para muchas decisiones que se tomaron en UML.

Este libro no pretende ser una guía sobre los documentos estándar de UML o sobre la estructura interna del metamodelo contenido en ellos. Los detalles del metamodelo son del interés de los metodólogos y los constructores de herramientas UML, pero la mayoría de los desarrolladores no necesitan de los misteriosos detalles de los documentos del Object Management Group (OMG). Este libro proporciona todos los detalles que necesitan la mayoría de los desarrolladores; en muchos casos, hace explícita información que, de lo contrario, habría que buscar entre líneas en los documentos originales. Para aquellos que quieran consultarlos, se han incluido los documentos originales, en inglés, en el CDROM adjunto.

Este libro está pensado como una referencia para aquellos que ya tienen algún conocimiento de la tecnología de orientación a objetos. Para los principiantes, en la bibliografía hay un listado de libros escritos por nosotros y por otros autores; aunque parte de la notación ha cambiado, libros como [Rumbaugh-91], [Booch-94], [Jacobson-92], y [Meyer-98 (edición en español)] proporcionan una introducción a los conceptos de orientación a objetos que todavía es válida y que por tanto no es necesario duplicar. Para una Introducción a UML que muestra como modelar varios problemas comunes, véase *Guía de Usuario del Lenguaje Unificado de Modelado* [Booch-99]. Aquellos que ya conocen un método orientado a objetos, como OMT, Booch, Objectory, Coad-Yourdon, o Fusion, deberían ser capaces de leer el *Manual de Referencia* y usarlo para entender la notación y la semántica de UML; para aprender UML rápidamente sin embargo, podrían encontrar útil leer la *Guía del Usuario*.

UML no requiere un proceso de desarrollo en particular, y este libro no describe ninguno. Aunque UML puede usarse con varios procesos de desarrollo, fue diseñado para usarse con un proceso iterativo, incremental, guiado por casos de uso y centrado en la arquitectura, el tipo de proceso que consideramos más apropiado para el desarrollo de sistemas complejos modernos. *El Proceso Unificado de Desarrollo de Software* [Jacobson-99] describe el tipo de proceso que creemos que complementa UML y que ayuda mejor al desarrollo de software.

Esquema general del libro

EL Manual de Referencia de UML está organizado en tres partes: una descripción de la historia de UML y del modelado, una visión de conjunto de los conceptos de UML, y una enciclopedia alfabética de los términos y conceptos de UML.

La primera parte es un examen del conjunto de UML —su historia, propósitos y usos— para ayudar a entender el origen de UML y las necesidades que intenta cubrir.

La segunda parte es una revisión de las vistas de UML para poner todos los conceptos en perspectiva. La visión de conjunto proporciona una breve descripción de las vistas disponibles en UML y muestra cómo interactúan las diferentes construcciones. Esta parte empieza con un ejemplo que abarca varias vistas de UML y también contiene un capítulo por cada clase de vista. Esta visión no pretende ser un curso completo ni una extensa descripción de conceptos. Sirve principalmente para resumir y relacionar los diferentes conceptos de UML y proporcionar puntos de partida para lecturas de la enciclopedia en detalle.

La tercera parte contiene el material de referencia organizado para acceder fácilmente a cada tema. La mayor parte del libro es una enciclopedia alfabética de todos los conceptos y construcciones de UML. Cada término de importancia en UML tiene su propia referencia en la enciclopedia. La enciclopedia debe ser completa, por lo que todos los conceptos descritos en la parte dos de este libro se repiten de forma más detallada. Algunas veces se ha incluido la misma o similar información en varios artículos de la enciclopedia, para que el lector pueda encontrarlas oportunamente.

La parte de referencia también contiene una lista alfabética de los elementos estándar de UML. Un elemento estándar es una característica predefinida que utiliza los mecanismos de extensión de UML. Los elementos estándar son extensiones pensadas para ser muy útiles.

Los apéndices muestran el metamodelo de UML, un resumen de la notación y algunos conjuntos estándar de extensiones para dominios particulares. Hay una breve bibliografía de los libros de orientación a objetos más importantes, pero no intenta ser una completa guía sobre las fuentes de ideas para UML y otros métodos. Muchos de los libros de la bibliografía contienen excelentes listas de referencias a libros y artículos para aquellos interesados en seguir el desarrollo de las ideas.

Convenciones en el formato de los artículos de la enciclopedia

La enciclopedia está organizada como una lista alfabética de entradas, cada una de las cuales describe un concepto en mayor o menor detalle. Los artículos representan una lista “plana” de los conceptos de UML a diferentes niveles conceptuales. Un concepto de alto nivel típicamente contiene un resumen de sus conceptos subordinados, cada uno de los cuales se describe completamente en un artículo separado. Los artículos tienen muchas referencias cruzadas. Esta organización “plana” de la enciclopedia permite presentar la descripción de cada concepto a un nivel bastante uniforme de detalle, sin los constantes desplazamientos de nivel que serían necesarios para una presentación secuencial de las descripciones anidadas. El formato de hiper-

texto también debería hacerlo útil como referencia. No debería ser necesario usar el índice; en su lugar vamos directamente al artículo principal de la enciclopedia para cualquier término de interés y seguimos las referencias cruzadas. Este formato no es necesariamente el ideal para aprender el lenguaje; se aconseja a los principiantes leer la descripción de UML que se encuentra en la Parte 2 o leer los libros introductorios sobre UML, tales como *El lenguaje Unificado de Modelado [Booch-99]*.

Los artículos de la enciclopedia tienen las siguientes divisiones (aunque no todas las divisiones aparecen en todos los artículos):

Breve definición

El nombre del concepto aparece en negrita, colocado a la izquierda del texto del artículo. A continuación una breve descripción en letra normal. Esta definición tiene la intención de capturar la idea principal del concepto, pero puede simplificar el concepto para su presentación concisa. Debemos remitirnos al artículo principal para significados precisos.

Semántica

Esta sección contiene una descripción detallada del significado del concepto, incluyendo las restricciones en su uso y las consecuencias de su ejecución. Esta sección no abarca la notación, aunque los ejemplos utilizan la notación apropiada: primero se da la semántica general. Para conceptos con propiedades estructurales subordinadas, a la semántica general sigue una lista de las propiedades, a menudo bajo el subtítulo de estructura. En la mayoría de los casos, las propiedades se presentan como una tabla ordenada alfabéticamente por nombre, con la descripción de cada propiedad a la derecha. Si una propiedad tiene una breve lista de opciones, aparecerán como una sublista sangrada. En casos más complicados, la propiedad tiene su propio artículo para evitar el excesivo anidamiento. Cuando las propiedades requieren más explicación que la permitida por una tabla, son descritas en texto normal con cabeceras en negrita y cursiva. En ciertos casos el concepto principal se describe de la mejor forma bajo varias subdivisiones lógicas en lugar de hacerlo en una lista. En tales casos, las secciones adicionales están a continuación o reemplazan a la estructura de la subsección. Aunque se han usado varios mecanismos de organización, su estructura debería ser obvia para el lector.

Notación

Esta sección contiene una descripción detallada del concepto. Normalmente la sección de notación tiene un formato semejante a la sección de semántica, cuya referencia, y a menudo tiene las mismas divisiones. La sección de notación a menudo incluye uno o varios diagramas para ilustrar el concepto.

Ejemplo

Esta subsección contiene ejemplos de notación o ilustraciones del uso del concepto. También es frecuente que los ejemplos traten situaciones complicadas o potencialmente confusas.

Discusión

Esta sección describe asuntos sutiles, clarifica los puntos difíciles y habitualmente confusos, y contiene otros detalles que, de lo contrario, serían apartados de una sección de semántica más descriptiva. Hay pocos artículos que tengan una sección de discusión.

Esta sección también explica ciertas decisiones de diseño hechas durante el desarrollo de UML, en particular aquellas que pueden parecer menos intuitivas o que han causado gran controversia. Sólo una pequeña parte de los artículos tienen esta sección. Las diferencias simples por gustos, generalmente no se contemplan.

Elementos estándar

Esta sección lista las restricciones, etiquetas, estereotipos, y otras convenciones estándar predefinidas para el concepto del artículo. Esta sección es muy poco común.

Convenciones de sintaxis

Expresiones sintácticas. Las expresiones sintácticas se dan en un formato BNF modificado escrito en un tipo de letra distinto.

Los subíndices y barras superiores se usan como operadores sintácticos de la siguiente manera:

$\text{expresión}_{\text{opc}}$	La expresión es opcional.
$\text{expresión}_{\text{lista}}$	Puede aparecer una lista de las expresiones separada por comas. Si hay cero o una repetición, no hay separador. Cada repetición puede tener una sustitución separada. Si aparece un signo de puntuación diferente a una coma en el subíndice, entonces es un separador.
$\overline{= \text{expresión}_{\text{opc}}}$	Una barra superior enlaza juntos uno o más términos que son considerados una unidad para la optatividad o la repetición de ocurrencias. En este ejemplo, el igual y la expresión forman una unidad que puede ser omitida o incluida. La barra superior es innecesaria si sólo hay un término.

Se evita que existan dos niveles de anidamiento.

Cadenas de literales. El texto ejecutable, las palabras clave del lenguaje, los nombres de elementos del modelo y las cadenas de ejemplo de modelos se muestran en un tipo de letra distinto.

Diagramas. En los diagramas, el texto y las flechas en azul son anotaciones, es decir, explicaciones de las notaciones de diagramas que no aparecen en un diagrama real. Cualquier texto y símbolos en tinta negra son notaciones de diagramas reales.

CD

Este libro se acompaña de un CD que contiene el texto completo en inglés del libro en formato “Adobe Acrobat Reader” (PDF). Usando “Adobe Acrobat Reader”, el lector puede buscar fácilmente en el libro una palabra o frase. La versión en CD también contiene una tabla de

contenidos cuyos términos son enlaces al texto donde aparecen definidos índice, miniaturas e hiperenlaces en el cuerpo de los artículos. Basta con pulsar en uno de los enlaces para saltar al artículo de la enciclopedia para esa palabra o frase.

El CD también contiene el texto completo de las especificaciones de OMG de UML, incluidas con el permiso del Object Management Group.

Tenemos la impresión de que este CD será una referencia muy útil para usuarios avanzados.

Información adicional

Pueden encontrarse archivos fuente adicionales e información actualizada sobre trabajos posteriores en UML y temas relacionados en los servidores Web www.rational.com y www.omg.org.

Reconocimientos

Queremos dar las gracias a todos aquellos que han hecho posible UML. Primero, debemos agradecer a Rational Software Corporation, especialmente a Mike Devlin y Paul Levy, quien tuvo la visión de reunirnos, empezar el trabajo de unificación, y continuar durante los cuatro años que hicieron falta para llevar el trabajo a un final con éxito. También queremos dar las gracias al Object Management Group por brindar el marco que reunió muchos puntos de vista y los fusionó en un amplio consenso lo que superó todas las otras contribuciones.

Queremos dar las gracias en particular a Cris Kobryn, quien lideró al equipo técnico que preparó el estándar de UML y que se las arregló para alcanzar un consenso entre un grupo de personas extremadamente exaltadas (y nosotros tres no fuimos el menor de sus problemas). Su habilidad diplomática y equilibrio técnico evitó que el esfuerzo de UML fracasara en medio de las muchas opiniones diferentes. Cris también revisó el libro y nos ofreció incontables y útiles sugerencias.

Queremos agradecer a Gunnar Övergaard por la revisión concienzuda del libro, así como su perseverancia en completar muchas secciones de los documentos de UML, lo cual no fue divertido de escribir pero sí necesario para su corrección formal.

Queremos dar las gracias a Karin Palmkvist por una revisión extremadamente detallista del libro que descubrió muchos errores en el contenido técnico así como varios fallos de gramática, expresión y presentación.

También nos gustaría agradecer a Mike Blaha, Conrad Bock, Perry Cole, Bruce Douglass, Martin Fowler, Eran Gery, Pete McBreen, Guus Ramackers, Tom Schultz, Ed Seidewitz, y Bran Selic por sus provechosas revisiones.

Sobre todo, queremos dar las gracias a los, incluso, cientos de personas que contribuyeron a la comunidad de ideas de las cuales derivó UML -ideas en tecnología de orientación a objetos, metodología de software, lenguajes de programación, interfaces de usuario, programación visual, y muchas otras áreas de la informática. Es imposible enumerarlas todas o, en realidad, incluso hacer un seguimiento de las cadenas de mayor influencia sin hacer un gran esfuerzo erudito, y este es un libro de ingeniería, no una reseña histórica. Muchos son muy conocidos pero muchas buenas ideas vienen de algunos que no tienen la buena suerte de ser ampliamente reconocidos.

A nivel más personal, quiero dar las gracias al Profesor Jack Dennis, quien inspiró mi trabajo en el modelado y el trabajo de muchos otros estudiantes, hace más de veinticinco años. Las ideas de su Grupo de Estructuras de Computación en el MIT han dado muchos frutos, y no son la menor de las fuentes de UML. También debo dar las gracias a Mary Loomis y Ashwin Shah, con quien desarrollé las ideas originales de OMT, y a mis primeros colegas en el Centro de I+D en GE, Mike Blaha, Bill Premerlani, Fred Eddy, y Bill Lorensen, con quien escribí el libro de OMT.

Finalmente sin la paciencia de mi esposa, Madeline, y mis hijos, Nick y Alex, no habría habido UML ni libro sobre él.

James Rumbaugh
Cupertino, California
Noviembre 1998



UML: Una historia del futuro del desarrollo de software

UML es, probablemente, una de las innovaciones conceptuales en el mundo tecnológico del desarrollo de software que más expectativas y entusiasmos haya generado en muchos años, comparable a la aparición e implantación de los lenguajes COBOL, BASIC, Pascal, C++, y más recientemente Java o XML. Además, todas las expectativas se han cumplido y han generado a su vez nuevas expectativas. UML es ya un *estándar de la industria*, pero no sólo de la industria del software sino, en general, de cualquier industria que requiera la construcción de modelos como condición previa para el diseño y posterior construcción de prototipos.

Desde la mítica versión 0.8 del entonces denominado *Unified Method* versión 0.8, fruto de la unión de los creadores de las metodologías Booch'94 y OMT (Grady Booch y James Rumbaugh) y que presentaron a la comunidad informática en la primavera de 1995. Posteriormente se unió al equipo Ivar Jacobson, creador a su vez del método OOSE y, sobre todo, del ingenioso concepto *use case* (casos de uso). La unión del equipo de "3 amigos", como se les ha conocido, (incluso se les ha comparado con *los 3 tenores* que en el campo de la música formaron Luciano Pavarotti, Plácido Domingo y José Carreras); 3 amigos con inmenso prestigio en el mundo de la ingeniería del software que se propusieron construir un nuevo lenguaje de modelado, UML, cuya primera versión (1.1) se presentó para su estandarización al OMG (Object Management Group) en 1997 y que fue aceptada. Las últimas versiones 1.2 y 1.3 se han presentado posteriormente y nos encontramos en período de evaluación y pruebas de la versión 1.4. Estas versiones están disponibles en el sitio Web de Rational (www.rational.com).

Otra gran aportación, en este caso no sólo conceptual sino práctica en forma de herramientas, fue la creación de una herramienta CASE (ingeniería de software asistida por computador) denominada Rational CASE, cuya versión Rational'98 está muy extendida en la industria y que sigue todas las especificaciones de UML. Recientemente se ha presentado Rational 2000, que ha mejorado sensiblemente respecto de Rational'98 y promete ser una de las herramientas de referencia en el mundo de la ingeniería y, en particular, en la ingeniería de software.

¿Por qué UML es tan influyente en la industria del software?

UML ha nacido como un lenguaje, pero es mucho más que un lenguaje de programación. Aunque en su génesis se parece a C++ o a Java, en realidad se ha diseñado y construido un lenguaje que ha nacido con una madurez muy acentuada si se le compara, incluso, con los últimos desarrollos de HTML, Java y XML, los lenguajes por excelencia del mundo Internet.

UML se ha diseñado realizando combinaciones de una gran cantidad de estándares, si bien se rige a través de tres metodologías procedentes de la colaboración de los tres creadores de UML ya citados, J. Rumbaugh, G. Booch e I. Jacobson, así como del análisis y estudio de alrededor de 20 métodos estándares que a su vez se han integrado en otro estándar, en este caso, UML; esta fue una gran iniciativa de los tres creadores que pusieron las especificaciones de UML a la consideración de la comunidad informática mundial, antes de su publicación. El diseño de UML ha sido completo desde el principio, al contrario que HTML que ha cambiado gradualmente, de forma que XML ha tratado de resolver los problemas de HTML y Java, que sigue todavía en el proceso de estandarización con la nueva versión 2.0. Al contrario que HTML/XML que son lenguajes de marcación (*markup*), UML es un lenguaje para modelar, que es el procedimiento que emplean los ingenieros para el diseño de software antes de pasar a su construcción, al igual que sucede con cualquier producto manufacturado o fabricado en serie.

UML ayuda al usuario a entender la realidad de la tecnología y la posibilidad de que reflexione antes de invertir y gastar grandes cantidades en proyectos que no estén seguros en su desarrollo, reduciendo el coste y el tiempo empleado en la construcción de las piezas que constituirán el modelo.

Sin embargo, desde el punto de vista puramente tecnológico, UML tiene una gran cantidad de propiedades que han sido las que, realmente, han contribuido a hacer de UML el estándar de facto de la industria que es en realidad. Algunas de las propiedades de UML como lenguaje de modelado estándar son:

- *Concurrencia, es un lenguaje distribuido y adecuado a las necesidades de conectividad actuales y futuras.*
- *Ampliamente utilizado por la industria desde su adopción por OMG.*
- *Reemplaza a decenas de notaciones empleadas con otros lenguajes.*
- *Modela estructuras complejas.*
- *Las estructuras más importantes que soportan tienen su fundamento en las tecnologías orientadas a objetos, tales como objetos, clase, componentes y nodos.*
- *Emplea operaciones abstractas como guía para variaciones futuras, añadiendo variables si es necesario.*
- *Comportamiento del sistema: casos de uso, diagramas de secuencia y de colaboraciones, que sirven para evaluar el estado de las máquinas.*

UML, la Web y el comercio electrónico

Uno de los grandes anuncios del año 2000 en el mundo de UML, se ha producido el pasado 17 de mayo con el anuncio conjunto de Rational Software (empresa creadora de UML y de Rational CASE) y Commerce One (líder mundial en soluciones globales de comercio electrónico) de colaboración mutua para crear la especificación de la primera versión UML para la industria, que siga las especificaciones XML para el desarrollo del comercio electrónico. Es decir, se han unido dos grandes empresas con el objeto de construir un método estándar que reduzca drásticamente el tiempo de desarrollo e incremente la calidad de las aplicaciones de comercio electrónico basadas en XML.

Se trata de alcanzar la máxima de que el cambio rápido de Internet ha creado una paradoja para el desarrollo del conocido como *e-software* para las organizaciones que requieren la entrega de software de un modo mucho más rápido pero conservando una calidad alta. La versión de UML para especificaciones XML está disponible en el sitio Web de Commerce One (www.commerceone.com/xml/sox/index.html) y en el sitio Web de Rational (www.rational.com/uml/index.jtmpl).

Otra gran ventaja que está ofreciendo UML se refiere al desarrollo de aplicaciones globales para la Web, no sólo para comercio electrónico. UML está siendo utilizado por los gerentes de proyectos, desarrolladores y arquitectos de la Web que aplican técnicas orientadas a objetos para construir aplicaciones Web robustas, escalables y eficientes. UML permite a los desarrolladores modelar sus aplicaciones Web como parte de un sistema completo y la lógica de negocios que se debe reflejar en las aplicaciones.

El futuro ya no es lo que era sin UML

UML, como se ha comentado, ha generado y sigue generando un enorme entusiasmo comparable al nacimiento del desarrollo orientado a objetos a principios de los 90 y posteriormente el desarrollo de componentes en la segunda mitad de la década. Este entusiasmo se ha hecho una gran realidad y UML se ha convertido ya en una de las mejores herramientas para el diseño y desarrollo de software fiable, eficiente y de calidad.

Buena prueba de que este entusiasmo es una realidad creciente, se puede ver en la inmensa cantidad de bibliografía que ha generado y sigue generando UML, así como en los numerosos eventos de todo tipo (congresos, seminarios, simposium, jornadas, etc.) que se celebran a lo largo y ancho del mundo tecnológico y en los que brilla con luz propia UML (*léase ECOOP, Tools, Object Expo, OOPSLA, UML Conference, etc.*).

UML está predestinado a convertirse en el lenguaje estándar de la industria para especificar, visualizar, construir y documentar sistemas de software del siglo XXI. Iniciativas como las acordadas con Commerce One, con Microsoft para incorporar UML como herramienta de diseño de Office 2000, con Sun para el desarrollo de Java, etc., hacen que UML sea el nuevo buque insignia de la industria del futuro.

Madrid, mayo de 2000
 Dr. Luis Joyanes Aguilar
Facultad de Informática
Universidad Pontificia de Salamanca campus de Madrid



UML permite modelar sistemas de información, y su objetivo es lograr modelos que, además de describir con cierto grado de formalismo tales sistemas, puedan ser entendidos por los clientes o usuarios de aquello que se modela. Para ello, es muy importante que el idioma en el que estén las palabras y textos que aparezcan en tales modelos sea el propio de estas personas.

En cualquier comunidad hispanohablante, leer *Pedido* aclara mucho más que ver *Order*, y leer *hereda de* más que leer *inherits*, porque los términos en español evocan directamente al lector una semántica cercana a la que se pretende con su uso en el modelo, y esa evocación es precisamente la razón por la cual fueron elegidos los términos ingleses de ese modelo. Como lenguaje de modelado y descripción, UML permite que *todo el modelo* se cree en español o en cualquier otro lenguaje. Sin embargo, el inglés ha sido el idioma nativo en la creación de UML, por lo que las **palabras clave** incorporadas en el propio UML están en inglés. Para el cliente y el equipo de desarrollo, esto no es un problema, pueden usar los términos traducidos que entenderán mejor; pero cuando se quiere usar una herramienta de ingeniería de software que entiende semántica UML, ésta esperará encontrar los términos ingleses para *entender* lo que quiere decir, por ejemplo, esa línea o figura.

Las palabras que aparecen en este Manual de Referencia de UML pueden clasificarse de cara a su traducción, al igual que las que aparezcan en cualquier documento UML, en varias categorías, del siguiente modo:

- **Palabras clave UML**, es decir, términos con un significado específico asignado por el propio UML, no por un modelo en particular, que **no** son extensiones UML (aplicaciones de los mecanismos de extensión de UML). Ejemplo: *send*, *entry*, etc.
- **Palabras clave que son extensiones UML**. Por ejemplo: `<<trace>>`, `{xor}`.
- **Extensiones de UML**, que *no son palabras clave UML* pero están incorporadas al propio estándar por ser de uso común. Éstas, junto con las palabras clave componen los **elementos estándar** de UML y están listadas en el Capítulo 14.
- **Extensiones de UML** que *no son elementos estándar*, sino extensiones introducidas por los modelos usados como ejemplo.
- **Palabras que forman parte de un modelo UML** en particular, usado como ejemplo.
- **Texto explicativo**, que en el libro es el texto básico de los párrafos; y en un modelo los textos de acompañamiento.

En el Capítulo 13 de este libro, cuyo uso será similar al de un diccionario de acceso rápido, se ha optado por mantener en inglés, en figuras y texto, los **elementos estándar** UML que

corresponden a las tres primeras categorías de la clasificación anterior. Estos términos, en los capítulos 1 al 12 se han traducido, como regla general, puesto que su lectura será probablemente más secuencial y pausada.

Para su más rápida localización, los títulos de los términos del Capítulo 13 se han traducido al castellano, mientras que los del Capítulo 14 se mantienen en inglés, pero haciendo referencia a los términos adecuados del Capítulo 13. De este modo, los **elementos estándar** se pueden localizar bien en inglés o bien en su traducción al castellano.

Cuando se crean modelos UML para su uso con herramientas de ingeniería de software que entiendan UML, puede ser necesario que las tres primeras categorías estén en inglés, según el grado de *comprensión* de UML que tenga la herramienta. Cuando los implicados en un modelo incluyen colegas de habla no hispana, todas las categorías, incluso las tres últimas, deben estar en el idioma convenido. En ausencia de las condiciones anteriores, para modelar sistemas en los que todos los implicados hablen castellano no es necesario utilizar términos ingleses en ninguna de las categorías de palabras de la clasificación anterior.

Madrid, mayo de 2000

Carmelo R. Fernández

Director de Sistemas de Información

Cámara de Comercio, Industria y Navegación de Gipuzkoa

Universidad del País Vasco / Euskal Herriko Unibersitatea

Parte 1: Antecedentes



Esta parte describe los principios generales subyacentes en UML, incluyendo la naturaleza y propósito del modelado y aquellos aspectos de UML que impregnan todas las áreas funcionales.

Este capítulo es una rápida descripción de UML y para qué es bueno.

Breve resumen de UML

El Lenguaje Unificado de Modelado (UML) es un lenguaje de modelado visual que se usa para especificar, visualizar, construir y documentar artefactos de un sistema de software. Captura decisiones y conocimiento sobre los sistemas que se deben construir. Se usa para entender, diseñar, hojear, configurar, mantener, y controlar la información sobre tales sistemas. Está pensado para usarse con todos los métodos de desarrollo, etapas del ciclo de vida, dominios de aplicación y medios. El lenguaje de modelado pretende unificar la experiencia pasada sobre técnicas de modelado e incorporar las mejores prácticas actuales en un acercamiento estándar. UML incluye conceptos semánticos, notación, y principios generales. Tiene partes estáticas, dinámicas, de entorno y organizativas. Está pensado para ser utilizado en herramientas interactivas de modelado visual que tengan generadores de código así como generadores de informes. La especificación de UML no define un proceso estándar pero está pensado para ser útil en un proceso de desarrollo iterativo. Pretende dar apoyo a la mayoría de los procesos de desarrollo orientados a objetos.

UML capta la información sobre la estructura estática y el comportamiento dinámico de un sistema. Un sistema se modela como una colección de objetos discretos que interactúan para realizar un trabajo que finalmente beneficia a un usuario externo. La estructura estática define los tipos de objetos importantes para un sistema y para su implementación, así como las relaciones entre los objetos. El comportamiento dinámico define la historia de los objetos en el tiempo y la comunicación entre objetos para cumplir sus objetivos. El modelar un sistema desde varios puntos de vista, separados pero relacionados, permite entenderlo para diferentes propósitos.

UML también contiene construcciones organizativas para agrupar los modelos en paquetes, lo que permite a los equipos de software dividir grandes sistemas en piezas de trabajo, para entender y controlar las dependencias entre paquetes, y para gestionar las versiones de las unidades del modelo, en un entorno de desarrollo complejo. Contiene construcciones para representar decisiones de implementación y para elementos de tiempo de ejecución en componentes.

UML no es un lenguaje de programación. Las herramientas pueden ofrecer generadores de código de UML para una gran variedad de lenguajes de programación, así como construir modelos por ingeniería inversa a partir de programas existentes. UML no es un lenguaje altamente formal pensado para probar teoremas. Hay varios lenguajes de ese tipo, pero no son fáciles de entender ni de usar para la mayoría de los propósitos. UML es un lenguaje de modelado de pro-

pósito general. Para dominios especializados, tales como la composición de IGU, diseño de circuitos VLSI, o inteligencia artificial basada en reglas, podría ser más apropiada una herramienta especializada con un lenguaje especial. UML es un lenguaje de modelado discreto. No se creó para modelar sistemas continuos como los basados en ingeniería y física. UML quiere ser un lenguaje de modelado universal, de propósito general, para sistemas discretos, tales como los compuestos por software, firmware o lógica digital.

Historia de UML

UML fue desarrollado en un esfuerzo para simplificar y consolidar el gran número de métodos de desarrollo orientado a objetos que habían surgido.

Los métodos de desarrollo orientado a objetos

Los métodos de desarrollo para los lenguajes de programación tradicionales, tales como Cobol y Fortran, emergieron en los años 70 y llegaron a ser ampliamente difundidos en los 80. Principalmente entre ellos estaba el Análisis estructurado y el diseño estructurado [Yourdon-79] y sus variantes tales como Diseño estructurado de tiempo real [Ward-85] y otros. Esos métodos originalmente desarrollados por Constantine, DeMarco, Mellor, Ward, Yourdon, y otros, alcanzaron cierta penetración en el área de los grandes sistemas, especialmente para los proyectos contratados por el gobierno en los campos aerospacial y de defensa, en los cuales los contratistas insistieron en un proceso de desarrollo organizado y en una amplia documentación del diseño e implementación del sistema. Los resultados no fueron siempre tan buenos como se esperaba—muchos sistemas de ingeniería de software asistidos por computador (CASE) fueron poco más que generadores de informes que extraían diseños después de que la implementación estuviera terminada—pero los métodos incluían buenas ideas que fueron usadas eficientemente en algunos casos para la construcción de grandes sistemas. Las aplicaciones comerciales fueron más reacias a adoptar grandes sistemas CASE y métodos de desarrollo. La mayoría de los negocios desarrollaban su software internamente según sus propias necesidades, sin la relación de enfrentamiento entre cliente y contratista que caracteriza los grandes proyectos del gobierno. Los sistemas comerciales se percibían como más simples, tanto si lo eran en verdad como si no, y por tanto había menos necesidad de una revisión por parte de una organización externa.

El primer lenguaje que es generalmente reconocido como orientado a objetos es Simula 67, desarrollado en 1967. Este lenguaje nunca tuvo un significativo seguimiento, aunque influyó notablemente en los desarrolladores de varios de los lenguajes orientados a objetos posteriores. El movimiento de la orientación a objetos se convirtió en activo con la amplia difusión de la disponibilidad del Smalltalk a principio de los 80, seguido por otros lenguajes orientados a objetos como Objective C, C++, Eiffel, y CLOS. El uso real de los lenguajes orientados a objetos fue limitado al principio, pero la orientación a objetos atrajo mucho la atención. Aproximadamente 5 años después de que Smalltalk llegara a ser conocido, fueron publicados los primeros métodos de desarrollo orientado a objetos por Shlaer/Mellor [Shlaer-88] y Coad/Yourdon [Coad-91], seguidos muy de cerca por libros de Booch [Booch-91], Rumbaugh/Blaha/Premerlani/Eddy/Lorensen [Rumbaugh-91], y Wirfs-Brock/Wilkerson/Wiener [Wirfs-Brock-90] (nótese que los años de los derechos editoriales, a menudo empiezan en julio del año anterior). Esos libros, unidos a los primeros libros de diseño de lenguajes de programación escritos por Goldberg/Robson [Goldberg-83], Cox [Cox-86], y Meyer [Meyer-88], iniciaron el campo de la metodología

orientada a objetos. La primera fase se completó al final de 1990. El libro de Objectory [Jacobson-92] fue publicado ligeramente después, basado en publicaciones anteriores. Este libro tomó un acercamiento un poco diferente con su enfoque sobre los casos de uso y el proceso de desarrollo.

Durante los siguientes cinco años, aparecieron muchos libros de metodologías orientadas a objetos, cada uno con su propio conjunto de conceptos, definiciones, notación, terminología y procesos. Algunos añadieron nuevos conceptos útiles, pero en general hubo gran similitud entre los conceptos propuestos por los diferentes autores. Muchos de los nuevos libros partieron de uno o más de los métodos existentes e hicieron extensiones o cambios menores. Los autores originales tampoco estuvieron inactivos; la mayoría de ellos actualizaron su trabajo original, a menudo incorporando buenas ideas de otros autores. En general, surgió un núcleo de conceptos comunes, junto con una gran variedad de conceptos aceptados por uno o dos autores pero no utilizados ampliamente. Incluso en el núcleo de conceptos comunes, hay pequeñas discrepancias entre los métodos que hacían una comparación detallada algo capciosa, especialmente para el lector ocasional.

Esfuerzo de unificación

Hubo algunos intentos tempranos de unificar los conceptos entre métodos. Un ejemplo destacable fue Fusion por Coleman y sus colegas [Coleman-94], el cual incluyó conceptos de OMT [Rumbaugh-91], Booch [Booch-91], y CRC [Wirfs-Brock-90]. Como no involucró a los autores originales, permaneció como otro nuevo método en lugar de un reemplazo de varios de los métodos existentes. El primer intento exitoso de combinar y reemplazar los métodos existentes llegó cuando Rumbaugh se unió a Booch en Rational Software Corporation en 1994. Ellos empezaron combinando conceptos de los métodos OMT y Booch, obteniendo como resultado una primera propuesta en 1995. En ese momento Jacobson también se unió a Rational y comenzó a trabajar con Booch y Rumbaugh. Su trabajo conjunto fue llamado Lenguaje Unificado de Modelado (UML). El impulso ganado al tener a los autores de tres de los métodos más importantes trabajando juntos para unificar sus enfoques desplazó la balanza en el campo de las metodologías orientadas a objetos, donde había habido muy poco incentivo (o al menos poca voluntad) de los metodólogos de abandonar algunos de sus propios conceptos para alcanzar la armonía.

En 1996, el Object Management Group (OMG) publicó una petición de propuestas para un enfoque estándar sobre el modelado orientado a objetos. Los autores de UML (Booch, Jacobson y Rumbaugh) empezaron a trabajar con metodólogos y desarrolladores de otras compañías, para generar una propuesta atractiva a los miembros de OMG, así como también un lenguaje de modelado, que sería ampliamente aceptado por los fabricantes de herramientas, metodólogos, y desarrolladores, quienes serían los usuarios eventuales. Empezaron también varios esfuerzos competitivos. Finalmente, todas las propuestas se unieron a la propuesta final de UML que fue sometida a consideración del OMG en septiembre de 1997. El producto final es una colaboración entre muchas personas. Nosotros empezamos el esfuerzo de UML y aportamos unos pocas buenas ideas, pero las ideas contenidas en él son el producto de muchas mentes.

Estandarización

El Lenguaje Unificado de Modelado fue adoptado unánimemente por los miembros de OMG como estándar en noviembre de 1997 [UML-98]. OMG asumió la responsabilidad de futuros de-

sarrollos en el estándar de UML. Incluso antes de que se adoptara finalmente, se publicaron varios libros esbozando los puntos clave de UML. Muchos proveedores de herramientas anunciaron apoyo o planes para ofrecer UML, y muchos metodólogos anunciaron que usarían la notación UML en sus trabajos futuros. El surgir de UML parece ser atractivo a la generalidad del público informático porque consolida la experiencia de varios autores con un estado oficial que reducirá las diferencias gratuitas entre herramientas. Creemos que la estandarización apoyará tanto la expansión del uso del modelado orientado a objetos entre los desarrolladores como la aparición de un robusto mercado en herramientas de formación y utilización, ahora que ni los usuarios ni los proveedores tienen que pensar qué metodologías usar y mantener.

Equipo principal

Las siguientes personas formaron parte del equipo principal de desarrollo de la propuesta de UML o trabajaron en el equipo de revisión:

Data Access Corporation: Tom Digre
 DHR Technologies: Ed Seidewitz
 HP: Martin Griss
 IBM: Steve Brodsky, Steve Cook, Jos Warmer
 I-Logix: Eran Gery, David Harel
 ICON Computing: Desmond D'Souza
 IntelliCorp and James Martin &Co.: Conrad Bock, James Odell
 MCI Systemhouse: Cris Kobryn, Joaquin Miller
 ObjectTime: John Hogg, Bran Selic
 Oracle: Guus Ramackers
 Platinum Technologies: Dilhar DeSilva
 Rational Software: Grady Booch, Ed Eykholt, Ivar Jacobson,
 Gunnar Övergaard, Karin Palmkvist, James Rumbaugh
 SAP: Oliver Wiegert
 SOFTEAM: Philippe Desfray
 Sterling Software: John Cheesman, Keith Short
 Taskon: Trygve Reenskaug
 Unisys: Sridhar Iyengar, GK Khalsa

¿Qué significa unificado?

La palabra unificado tiene los siguientes significados relevantes para UML.

A través de los métodos históricos y notaciones. UML combina conceptos comúnmente aceptados por muchos métodos orientados a objetos, seleccionando una definición clara para cada concepto, así como una notación y una terminología. UML puede representar la mayoría de los modelos existentes tan bien o mejor que como lo hacían los métodos originales.

A través del ciclo de vida de desarrollo. UML no tiene saltos ni discontinuidades desde los requisitos a la implantación. Se puede utilizar el mismo conjunto de conceptos y notación en las diferentes etapas del desarrollo, incluso mezcladas en un solo modelo. No es necesario traducir de una etapa a otra. Esta continuidad es crítica para un desarrollo iterativo e incremental.

A través de los dominios de aplicación. UML está pensado para modelar la mayoría de los dominios de aplicación, incluyendo aquellos que implican sistemas grandes, complejos, de tiempo real, distribuidos, con tratamiento intensivo de datos o con cálculo intensivo, entre otras propiedades. Puede haber áreas especializadas en las cuales un lenguaje especial para ese propósito resulte más útil, pero UML pretende ser tan bueno o mejor que cualquier otro lenguaje de modelado de propósito general para la mayoría de las áreas de aplicación.

A través de los lenguajes de implementación y plataformas. UML está pensado para ser usado en sistemas desarrollados en varios lenguajes de implementación y plataformas, incluyendo lenguajes de programación, bases de datos, 4GLs, documentos de organización, firmware, y otros. El trabajo de la capa superior debería ser idéntico o similar, mientras que el trabajo de la capa inferior diferirá en algo para cada medio.

A través de procesos de desarrollo. El UML es un lenguaje, no una descripción de un proceso de desarrollo detallado. Se pretende que sea usado como lenguaje de modelado subyacente a la mayoría de los procesos de desarrollo existentes o de nueva creación, de la misma forma que un lenguaje de programación de propósito general puede ser usado en varios estilos de programación. Está especialmente pensado para apoyar un estilo de desarrollo iterativo e incremental, que es el que recomendamos.

A través de los conceptos internos. En la construcción del metamodelo de UML, hicimos un esfuerzo deliberado para descubrir y representar las relaciones subyacentes entre varios conceptos, intentando captar conceptos de modelado de manera abierta, aplicable a muchas situaciones conocidas y desconocidas. Este proceso permitió comprender mejor los conceptos y hacerlos más aplicables. Éste no fue el propósito original de la unificación, pero sí uno de los resultados más importantes.

Objetivos de UML

Hubo varios objetivos detrás del desarrollo de UML. El primero y más importante, UML es un lenguaje de modelado de propósito general que pueden usar todos los modeladores. No tiene propietario y está basado en el común acuerdo de gran parte de la comunidad informática. Esto significa incluir conceptos de los métodos líderes para que UML pueda usarse como su lenguaje de modelado. Está pensado para reemplazar al menos los modelos de OMT, Booch y Objectory, así como aquéllos de otros participantes de la propuesta. Se pensó para ser tan familiar como sea posible, usar la notación de OMT, Booch, Objectory y otros métodos importantes. Esto significa incorporar buenas prácticas de diseño, tales como la encapsulación, separación de los temas, y la captura de la intención del modelo construido. Pretende abordar los problemas actuales del desarrollo de software, tales como gran tamaño, distribución, concurrencia, patrones, y desarrollo en equipo.

UML no pretende ser un método de desarrollo completo. No incluye un proceso de desarrollo paso a paso. Creemos que un buen proceso de desarrollo es crucial para el éxito de un desarrollo de software, y proponemos uno en un libro complementario [Jacobson-99]. Es importante darse cuenta de que UML y el proceso para usar UML son dos cosas independientes.

UML pretende trabajar correctamente con todos, o al menos con la mayoría de los procesos de desarrollo existentes. UML incluye todos los conceptos que consideramos necesarios para utilizar un proceso moderno iterativo, basado en construir una sólida arquitectura para resolver requisitos dirigidos por casos de uso.

Un objetivo final de UML era ser tan simple como fuera posible pero manteniendo la capacidad de modelar toda la gama de sistemas que se necesita construir. UML necesita ser lo suficientemente expresivo para manejar todos los conceptos que se originan en un sistema moderno, tales como la concurrencia y distribución, así como también los mecanismos de la ingeniería de software, tales como encapsulación y componentes. Debe ser un lenguaje universal, como cualquier lenguaje de programación de propósito general. Desafortunadamente eso significa que no puede ser pequeño si quiere manejar cosas que no sean sistemas de juguete. Los lenguajes modernos y los sistemas operativos modernos, son mucho más complicados hoy que hace 40 años, porque nosotros esperamos mucho más de ellos. UML tiene varios tipos de modelos; no es algo que uno pueda dominar en un día. Es más complicado que algunos de sus antecesores porque intenta ser más amplio. Pero no es necesario aprenderlo todo a la vez, no más que lo que exige un lenguaje de programación, un sistema operativo, o una compleja aplicación de usuario.

Áreas conceptuales de UML

Los conceptos y modelos de UML pueden agruparse en las siguientes áreas conceptuales.

Estructura estática. Cualquier modelo preciso debe primero definir el universo del discurso, esto es, los conceptos clave de la aplicación, sus propiedades internas, y las relaciones entre cada una. Este conjunto de construcciones es la vista estática. Los conceptos de la aplicación son modelados como clases, cada una de las cuales describe un conjunto de objetos discretos que almacenan información y se comunican para implementar un comportamiento. La información que almacenan es modelada como atributos; el comportamiento que realizan es modelado como operaciones. Varias clases pueden compartir una estructura común usando generalización. Una clase hija añade nuevas estructuras y comportamientos a las estructuras y comportamientos que obtiene por herencia de la clase padre. Los objetos también tienen conexión durante la ejecución con otros objetos individuales. Tales relaciones “Objeto a Objeto” son modeladas como asociaciones entre clases. Algunas relaciones entre elementos se agrupan como relaciones de dependencia, incluyendo las relaciones para modelar desplazamientos en los niveles de abstracción, enlace de parámetros del modelo, concesión de permisos, y uso de un elemento por parte de otro. Otras relaciones son la combinación de casos de uso y el flujo de datos. La vista estática se expresa con diagramas de clases y puede usarse para generar la mayoría de las declaraciones de estructuras de datos en un programa. Hay muchos tipos de elementos en los diagramas UML, tales como interfaces, tipos de datos, casos de uso y señales. En conjunto, se les llama clasificadores, y se comportan de forma muy parecida a las clases, con ciertas restricciones en cada tipo de clasificador.

Comportamiento dinámico. Hay dos formas de modelar el comportamiento. Una es la historia de la vida de un objeto, que muestra la forma en que interactúa con el resto del mundo; la otra son los patrones de comunicación de un conjunto de objetos conectados, que muestra cómo interactúan para implementar su comportamiento.

La visión de un objeto aislado es una máquina de estados —una vista de un objeto que muestra la forma en que responde a los eventos en función de su estado actual, realiza acciones

como parte de su respuesta y transiciones a un nuevo estado—. Las máquinas de estados se representan en un diagrama de estados.

La visión de la interacción de los objetos de un sistema es una colaboración: una vista, dependiente de contexto, de los objetos y los enlaces entre ellos, junto con el flujo de mensajes entre los objetos mediante los enlaces de datos. Este punto de vista unifica la estructura de los datos, el control de flujo y el flujo de datos en una sola vista. Las colaboraciones e interacciones se muestran mediante los diagramas de secuencia y los diagramas de colaboración. Guiando todas las vistas de comportamiento se encuentra un conjunto de casos de uso. Cada uno es una descripción de una porción de la funcionalidad del sistema como la percibe un actor, un usuario externo del sistema.

Construcciones de implementación. Los modelos de UML tienen significado para el análisis lógico y para la implementación física. Ciertos constructores representan elementos de implementación. Un componente es una parte física reemplazable de un sistema y es capaz de responder a las peticiones descritas por un conjunto de interfaces. Se pretende que sea fácilmente sustituible por otro componente que cumpla la misma especificación. Un nodo es un recurso computacional que define una localización durante la ejecución del sistema. Pueden contener componentes y objetos. La vista de despliegue describe la configuración de los nodos de un sistema en ejecución y la organización de los componentes y objetos en él, incluyendo posibles migraciones de contenido entre nodos.

Organización del modelo. Los ordenadores pueden manejar grandes modelos “planos”, pero los humanos no. En un sistema grande, la información del modelo debe ser dividida en piezas coherentes, para que los equipos puedan trabajar en las diferentes partes de forma concurrente. Incluso en un sistema más pequeño, el conocimiento humano requiere que se organice el contenido del modelo en paquetes de tamaño modesto. Los paquetes son unidades organizativas, jerárquicas, y de propósito general de los modelos de UML. Pueden usarse para almacenamiento, control de acceso, gestión de la configuración, y construcción de bibliotecas que contengan fragmentos de código reutilizable. Una dependencia entre paquetes resume las dependencias entre los contenidos del paquete. Una dependencia entre paquetes puede ser impuesta por la arquitectura global del sistema. Por lo tanto, los contenidos de los paquetes deben adaptarse a las dependencias del paquete y a las impuestas por la arquitectura del sistema.

Mecanismos de extensión. No importa cuan completo sea el conjunto de “facilidades” de un lenguaje, la gente querrá hacer extensiones. Hemos dotado a UML de una limitada capacidad de extensión, que creemos suficiente para la mayoría de las extensiones que requiere el “día a día”, sin la necesidad de un cambio en el lenguaje básico. Un estereotipo es una nueva clase de elemento de modelado con la misma estructura que un elemento existente pero con restricciones adicionales, una interpretación diferente de un icono, y un tratamiento diferente por los generadores de código y otras herramientas de bajo nivel. Un valor etiquetado es un par arbitrario de cadenas etiqueta-valor, que pueden enlazarse a cualquier tipo de elemento de modelado, para almacenar información arbitraria, como información de gestión de proyecto, guías para los generadores de código, y valores requeridos por los estereotipos. La etiqueta y el valor son representadas como cadenas. Una restricción es una condición “bien formada” expresada en una cadena de texto en algún lenguaje restringido, tal como un lenguaje de programación, un lenguaje especial limitado, o lenguaje natural. UML incluye un lenguaje de restricciones llamado OCL¹. Como con cualquier mecanismo de extensión, estos mecanismos deben usarse con cui-

¹ Object Constraint Language, “Lenguaje de Restricción de Objetos”, N. del T.

dado debido al riesgo de producir un dialecto privado ilegible por los demás. Pero a la vez pueden evitar la necesidad de cambios más radicales.

Sintaxis de los diagramas y las expresiones

Este libro contiene expresiones y diagramas que muestran ejemplos de modelos reales, así como también la sintaxis de las expresiones y anotaciones explicando los diagramas. Para reducir el peligro de confundir las explicaciones con los ejemplos, se han usado ciertas convenciones de formato.

Dentro de los diagramas y las expresiones de texto, los fragmentos de diagrama o literales de texto, que aparecerán en la notación real, se muestran en letra “Helvética”. Por ejemplo, un nombre de clase en negro es un nombre legal que puede aparecer en un modelo. Un paréntesis en una expresión de sintaxis es un paréntesis literal que aparecerá en la expresión real; no es parte de la “maquinaria” sintáctica. Por ejemplo:

Pedido.crear (cliente,cantidad)

Dentro del texto ejecutable, las palabras literales del modelo y los nombres de los elementos del modelo, también se muestran en letra “Helvética” negra, como **Pedido** o **cliente**.

En una expresión sintáctica, los nombres de las unidades sintácticas, que deben ser reemplazadas por las expansiones reales del texto, se muestran en letra “Helvética” azul, como nombre.

La aparición de texto negro en una expresión, representa un valor literal que aparecerá en la notación destino. El uso de cursivas o subrayado significa, que el texto que la reemplazará, tiene esa propiedad. Por ejemplo:

nombre.operación (argumento...)

nombre-objeto: clase

En una expresión sintáctica, un subíndice azul y una barra superior azul, se usan para denotar ciertas propiedades sintácticas.

expresión_{opc} La expresión es opcional.

expresión_{lista}, Puede aparecer una lista de expresiones separadas por comas. Si hay cero o una repetición, no hay separador. Cada repetición puede tener una sustitución independiente. Si aparece un signo de puntuación diferente de la coma, éste es el separador.

expresión_{opc} Una barra superior enlaza juntos uno a más términos, que son considerados una unidad para la optatividad o la repetición de ocurrencias. En este ejemplo, el igual y la expresión forman una unidad que puede ser omitida o incluida. La barra superior es innecesaria si sólo hay un término.

En un diagrama los textos y flechas en azul son anotaciones. No son parte de la notación real, pero se pensaron como explicaciones. Los símbolos y texto en negro son parte de la notación.



La naturaleza y propósito de los modelos

Este capítulo explica qué son los modelos, para qué son buenos, y cómo se usan. También explica varias clases de modelos: ideal, parcial y basado en herramientas.

¿Qué es un modelo?

Un modelo es una representación, en cierto medio, de algo en el mismo u otro medio. El modelo capta los aspectos importantes de lo que estamos modelando, desde cierto punto de vista, y simplifica u omite el resto. La ingeniería, la arquitectura y muchos otros campos creativos usan modelos.

Un modelo se expresa en un medio adecuado para el trabajo. Los modelos de construcciones pueden dibujarse en papel, las figuras tridimensionales son construidas con cartón y papel cartón, o las ecuaciones de elementos finitos en un computador. Un modelo de construcción de un edificio muestra la apariencia del edificio pero también puede usarse para hacer ingeniería y cálculos de coste.

Un modelo de un sistema software está construido en un lenguaje de modelado, como UML. El modelo tiene semántica y notación y puede adoptar varios formatos que incluyen texto y gráficos. El modelo pretende ser más fácil de usar para ciertos propósitos que el sistema final.

¿Para qué sirven los modelos?

Los modelos se usan para muchos propósitos.

Para captar y enumerar exhaustivamente los requisitos y el dominio de conocimiento, de forma que todos los implicados puedan entenderlos y estar de acuerdo con ellos. Diferentes modelos de un edificio captan requisitos sobre la apariencia, los patrones de tráfico, varios tipos de servicios, fortaleza frente al viento y terremotos, costes y otras cosas. Los implicados incluyen al arquitecto, aparejador, contratista, varias subcontratas, el dueño, los inquilinos y la ciudad.

Los diversos modelos de un sistema de software pueden capturar requisitos sobre su dominio de aplicación, las formas en que los usuarios lo utilizarán, su división en módulos, los patrones comunes usados en su construcción, y otras cosas. Los implicados incluyen al arquitecto, a los analistas, a los programadores, al encargado del proyecto, a los clientes, a inversores, a los usuarios finales, y a los operadores. Se utilizan los diferentes tipos de modelos de UML.

Para pensar del diseño de un sistema. Un arquitecto utiliza modelos en papel, sobre un computador, o como construcciones tridimensionales, para visualizar y experimentar con posibles diseños. La simplicidad de crear y de modificar modelos pequeños permite pensamiento creativo e innovación con poco coste.

Un modelo de un sistema software ayuda a los desarrolladores a explorar varias arquitecturas y soluciones de diseño fácilmente, antes de escribir el código. Un buen lenguaje de modelado permite que el diseñador consiga la arquitectura correcta antes de que comience el diseño detallado.

Para capturar decisiones del diseño en una forma mutable a partir de los requisitos. Un modelo de un edificio muestra el aspecto externo convenido con el cliente. Otro modelo muestra el encaminamiento interno de cables, de tuberías, y de conductos de ventilación. Hay muchas maneras de implementar estos servicios. El modelo final muestra un diseño que el arquitecto cree que es bueno. El cliente puede verificar esta información, pero, a menudo, los clientes no se preocupan por los detalles mientras funcionan.

Un modelo de un sistema software puede captar el comportamiento externo de un sistema y la información del dominio del mundo real representado por el sistema. Otro modelo muestra las clases y las operaciones internas, que implementan el comportamiento externo. Hay muchas maneras de implementar el comportamiento; el modelo final de diseño muestra un acercamiento que el diseñador cree correcto.

Para generar productos aprovechables para el trabajo. Un modelo de un edificio se puede utilizar para generar varias clases de productos. Éstos incluyen una cuenta de materiales, una simulación animada de un paseo, una tabla de desviaciones a varias velocidades del viento, o una visualización de la tensión en varios puntos en el armazón.

Un modelo de un sistema software se puede utilizar para generar las declaraciones de clase, los cuerpos de procedimiento, las interfaces de usuario, las bases de datos, los escenarios de uso válidos o los guiones de configuración.

Para organizar, encontrar, filtrar, recuperar, examinar, y corregir la información en grandes sistemas. Un modelo de un edificio organiza la información por servicio: estructural, eléctrico, fontanería, ventilación, decoración, etcétera. A menos que el modelo esté en un computador, no es tan fácil encontrar cosas y modificarlas. Si está en un computador, los cambios se pueden realizar y recordar fácilmente, y los múltiples diseños pueden ser explorados fácilmente mientras comparten algunos elementos comunes.

Un modelo de un sistema de software organiza la información en varias vistas: estructura estática, máquinas de estado, interacciones, requisitos, etc. Cada vista es una proyección del modelo completo para un propósito determinado. Mantener un modelo, de cualquier tamaño, es imposible sin tener una herramienta de edición, que maneje el modelo. Un editor gráfico e interactivo del modelo, puede presentar la información en diversos formatos, ocultar información que es innecesaria para un propósito dado, y mostrarla otra vez más adelante, agrupar operaciones relacionadas, realizar cambios en los elementos individuales, así como cambiar grupos de elementos con un comando, etc.

Para explorar económicamente múltiples soluciones. Las ventajas y los riesgos de diversos métodos de diseño para edificios, pueden no estar claros al principio. Por ejemplo, diversas subestructuras pueden interactuar de formas complicadas, que no se pueden evaluar en la cabeza

de un ingeniero. Los modelos pueden explorar los diversos diseños y permitir cálculos de costes y de riesgos, antes de que se construya el edificio real.

Los modelos de un sistema de software grande permiten que se propongan y comparen varios diseños. Los modelos no se construyen al detalle, por supuesto, pero incluso un modelo rudimentario puede exponer muchas cuestiones que el diseño final debe tener en cuenta. Modelar permite considerar varios diseños, con un coste pequeño al implementar cualquiera de ellos.

Para domesticar los sistemas complejos. Un modelo de ingeniería de un tornado acercándose a un edificio, proporciona un conocimiento que no es posible obtener de un edificio del mundo real. Un tornado verdadero no se puede producir a voluntad, y destruiría los instrumentos de medida, de todas formas. Muchos procesos físicos rápidos, pequeños, o violentos se pueden entender usando modelos físicos.

Un modelo de un sistema software grande permite ocuparse de la complejidad que es demasiado difícil de tratar directamente. Un modelo se puede abstraer a un nivel que sea comprensible a los seres humanos sin perderse en detalles. Un computador puede realizar complicados análisis en un modelo, en un esfuerzo para encontrar problemas tales como errores de sincronización o desbordamiento en los recursos. Un modelo puede determinar el impacto potencial de un cambio antes de que se haga, explorando dependencias en el sistema. Un modelo puede también mostrar cómo reestructurar un sistema para reducir tales efectos.

Niveles de los modelos

Los modelos adquieren diversas formas para diferentes propósitos, y aparecen en diversos niveles de abstracción. La cantidad de detalle del modelo debe adaptarse a uno de los siguientes propósitos:

Guías al proceso de pensamiento. Los modelos de alto nivel contruidos al principio de un proyecto, sirven para enfocar el proceso del pensamiento de los participantes y destacar determinadas opciones. Capturan requisitos y representan un punto de partida hacia un diseño del sistema. Los primeros modelos ayudan a los autores a explorar las opciones posibles antes de converger en un concepto de sistema. Según progresa el diseño, los primeros modelos son sustituidos por otros modelos más exactos.

No hay necesidad de conservar cada giro y vuelta del proceso de exploración inicial. Su propósito es producir ideas. Sin embargo los modelos de pensamiento finales, se deben preservar incluso después de que nuestro foco de atención se desplace a tareas de diseño. Los primeros modelos no requieren el detalle o la precisión de un modelo de implementación, y no necesitan una gama completa de conceptos de implementación. Tales modelos utilizan un subconjunto de construcciones de UML, un subconjunto más limitado que los de modelos posteriores.

Cuando un primer modelo es una vista completa de un sistema con determinada precisión—por ejemplo, el modelo resultante del análisis de lo que debe ser hecho— debe ser preservado cuando el desarrollo pasa a la etapa siguiente. Hay una diferencia importante entre añadir detalles (en cuyo caso la cadena de razonamiento debe ser preservada) y el proceso normal de caminar al azar, explorando muchos callejones sin salida, antes de llegar a la solución correcta. En este último caso es generalmente abrumador e innecesario guardar la historia completa, excepto en las situaciones extraordinarias, en las cuales se requiere la capacidad completa de averiguar lo ocurrido.

Especificaciones abstractas de la estructura esencial de un sistema. Los modelos en el análisis o las etapas preliminares del diseño se centran en los conceptos y mecanismos claves del probable sistema. Se corresponden de cierta manera con el sistema final. Pero faltan los detalles, que se deben agregar explícitamente durante el proceso de diseño. El propósito de los modelos abstractos es conseguir que los aspectos de alto nivel estén correctos, antes de abordar los detalles más localizados. Estos modelos se piensan para evolucionar en los modelos finales, mediante un proceso cuidadoso que garantice que el sistema final implementa correctamente el objetivo de los modelos anteriores. Estos modelos esenciales deben corresponderse con los modelos completos; sino, no hay seguridad de que el sistema final incorpore correctamente las características clave, que el modelo esencial intentó mostrar. Los modelos esenciales se centran en la semántica. No necesitan la gama completa de opciones de implementación. De hecho, los detalles del funcionamiento a bajo nivel oscurecen a menudo la semántica lógica. La trayectoria de un modelo esencial a un modelo completo de implementación debe ser clara y directa, no importa si ha sido generada automáticamente por un generador de código o desarrollada manualmente por un diseñador.

Especificaciones completas de un sistema final. Un modelo de implementación incluye suficiente información para construir el sistema. Debe incluir, no solamente la semántica lógica del sistema y los algoritmos, las estructuras de datos y los mecanismos que aseguran funcionamiento apropiado, sino también las decisiones de organización sobre los artefactos del sistema que son necesarios, permitiendo así el trabajo cooperativo de las personas y el procesamiento por parte de las herramientas. Esta clase de modelo debe incluir las construcciones para empaquetar el modelo, para la comprensión de la persona, y para la conveniencia de la computadora. Éstas no son las características de la aplicación en sí misma. En realidad, son características del proceso de construcción.

Ejemplos de sistemas típicos o posibles. Algunos bien elegidos pueden facilitar el entendimiento a las personas, y pueden validar las especificaciones e implementación del sistema. Una gran colección de ejemplos, sin embargo, no elimina la necesidad de una descripción definitiva. En última instancia necesitamos los modelos que especifican el caso general; que es un programa, después de todo. Sin embargo, los ejemplos de las estructuras de datos típicas, de las secuencias de interacción, o de las historias de los objetos, pueden ayudar a las personas a intentar entender una situación complicada. Los ejemplos se deben utilizar con un cierto cuidado. Es lógicamente imposible inducir el caso general, a partir de un conjunto de ejemplos, pero los prototipos bien elegidos son la manera de pensar de la mayoría de la gente. Un modelo de ejemplo incluye instancias más que descriptores generales y por lo tanto, tiende a dar una visión diferente de la que da un modelo descriptivo genérico. Los modelos de ejemplo generalmente usan sólo un subconjunto de las construcciones de UML, las que se ocupan de instancias. Los modelos descriptivos y los modelos de ejemplo son útiles para modelar un sistema.

Descripciones completas o parciales de sistemas. Un modelo puede ser una descripción completa de un solo sistema, sin referencias externas. Más a menudo se organiza como un conjunto de unidades distintas, discretas, cada una de las cuales se puede almacenar y manipular por separado, como parte de la descripción completa. Tales modelos tienen conexiones que se deben enlazar a otros modelos en un sistema completo. Como las piezas tienen coherencia y significado, pueden ser combinadas con otras piezas de varias maneras para producir sistemas muy diversos. Lograr la reutilización es una meta importante del buen modelado.

Los modelos cambian con el tiempo. Los modelos con mayor grado de detalle se derivan de modelos más abstractos, y los modelos más concretos se derivan de modelos más lógicos. Por

ejemplo, un modelo pudo comenzar como vista de alto nivel del sistema entero, con algunos servicios clave poco detallados y ningún adorno. En un cierto plazo, se agrega mucho más detalle y se introducen las variaciones. También en un cierto plazo, el enfoque cambia de una visión centrada en el usuario y el aspecto exterior del sistema a una visión lógica centrada en su núcleo. A medida que los desarrolladores trabajan con un sistema y lo entienden mejor, el modelo se debe iterar en todos los niveles para capturar ese conocimiento: es imposible entender un sistema grande en una sola pasada lineal. No hay ninguna forma “correcta” para un modelo.

¿Qué hay en un modelo?

Semántica y presentación. Los modelos tienen dos aspectos importantes: Información semántica (semántica) y presentación visual (notación).

El aspecto semántico capta el significado de una aplicación como una red de construcciones lógicas, por ejemplo clases, asociaciones, estados, casos de uso, y mensajes. Los elementos semánticos del modelo llevan el significado del modelo, es decir, transportan la semántica. Los elementos semánticos del modelo se utilizan para la generación del código, la comprobación de la validez, las métricas de complejidad, etc. El aspecto visual es irrelevante para la mayoría de las herramientas que procesan modelos. La información semántica a menudo es llamada *el modelo*. Un modelo semántico tiene una estructura sintáctica, reglas para asegurar su corrección, y dinámicas de ejecución. Estos aspectos se describen a menudo por separado (como en los documentos de definición de UML), pero se correlacionan firmemente y parten de un solo modelo coherente.

La presentación visual muestra la información semántica de modo que se pueda ser considerada, hojeada y corregida por los seres humanos. Los elementos de la presentación llevan la presentación visual del modelo —esto es, lo muestran en una forma directamente comprensible por las personas—. No agregan significado, sino que organizan la presentación, para accentuar la organización del modelo de una manera útil. Por lo tanto, dirigen la comprensión humana del modelo. Los elementos de la presentación derivan su semántica de elementos semánticos del modelo. Pero, ya que la disposición de los diagramas la realizan las personas, los elementos de la presentación no son totalmente derivables de elementos lógicos. La disposición de los elementos de presentación puede contener connotaciones sobre las relaciones semánticas que son demasiado débiles o ambiguas para ser formalizadas en el modelo semántico pero que, sin embargo, sugieren algo a las personas.

Contexto. Los modelos son artefactos en un sistema informático, y se utilizan dentro de un contexto más grande que les dé significado completo. Este contexto incluye la organización interna del modelo, anotaciones sobre el uso de cada modelo en el proceso total del desarrollo, un sistema de valores por defecto y de suposiciones para la creación y la manipulación del elemento, y una relación al entorno en el cual se utilizan.

Los modelos requieren una organización interna que permita su uso simultáneo por varios grupos de trabajo, sin interferencias indebidas. Esta descomposición no se hace necesaria por razones semánticas —un modelo monolítico grande sería tan exacto como un sistema de modelos organizados en los paquetes coherentes, quizá aún más exacto porque los límites de organización complican el trabajo de definir la semántica exacta—. Pero los equipos de trabajadores no podrían trabajar con eficacia en un modelo monolítico grande sin entorpecerse constantemente unos a otros. Por otra parte, un modelo monolítico no tiene piezas que se puedan reutilizar en otras situaciones. Finalmente, los cambios en un modelo grande tienen consecuencias difíciles de de-

terminar. Los cambios en una pieza pequeña, aislada, de un modelo grande pueden ser manejables si el modelo se estructura correctamente en subsistemas con interfaces bien definidas. En cualquier caso, dividir los sistemas grandes en una jerarquía de piezas bien elegidas es la manera más fiable de diseñar sistemas grandes que los seres humanos han inventado en miles de años.

Los modelos capturan la información semántica sobre un sistema, pero también necesitan registrar mucha información sobre el proceso de desarrollo mismo, tal como el autor de una clase, el estado de depuración de un procedimiento, y el permiso de acceso de una persona a un diagrama. Tal información es, en el mejor de los casos, ajena a la semántica del sistema, pero es importante para el proceso de desarrollo. El modelo de un sistema, por lo tanto, necesita incluir ambos puntos de vista. Esto se consigue más fácilmente añadiendo información de gestión del proyecto, como anotaciones al modelo semántico (descripciones arbitrarias añadidas a los elementos del modelo pero cuyo significado está fuera del lenguaje de modelado). En UML se implementan como cadenas de texto.

Los comandos usados para crear y modificar un modelo no son parte de la semántica del lenguaje de modelado, del mismo modo en que los comandos de un editor de textos o de un navegador no son parte de la semántica de un lenguaje de programación. Las propiedades del elemento del modelo no tienen valores *prefijados*; en un modelo particular, simplemente tienen *valores*. Para el desarrollo práctico, sin embargo, las personas necesitan construir y modificar modelos, sin tener que especificar todo completo en detalle. Los valores prefijados están en el límite entre el lenguaje de modelado y la herramienta de edición que lo soporta. Son realmente valores por defecto en los comandos de la herramienta que crea un modelo aunque pueden trascender de una herramienta individual y convertirse en expectativas del usuario sobre la implementación del lenguaje por las herramientas en general.

Los modelos no se construyen y utilizan aislados. Son parte de un entorno más grande que incluye herramientas de modelado, lenguajes y compiladores, sistemas operativos, redes de computadores, restricciones de implementación, etc. La información sobre un sistema incluye la información sobre todas las partes del entorno. Parte será almacenado en un modelo aunque no es información semántica; por ejemplo, las anotaciones de gestión de proyecto (discutidas antes), las ayudas y directivas de la generación de código, empaquetado del modelo, y la configuración por defecto para herramienta de edición. El resto se puede almacenar por separado: código fuente del programa, comandos de configuración del sistema operativo, etc. Incluso si una cierta información es parte de un modelo, la responsabilidad de interpretarlo puede subyacer en varios lugares, incluyendo el lenguaje de modelado, la herramienta de modelado, el generador de código, el compilador, un lenguaje de comandos, etc. Este libro describe la interpretación de modelos hechos con el lenguaje UML, pero cuando funcionan en un entorno físico de desarrollo, otras fuentes pueden agregar interpretaciones adicionales a la información que es opaca para UML.

¿Cuál es el significado de un modelo?

Un modelo es un *generador* de potenciales configuraciones de sistemas; los posibles sistemas son sus *extensiones*, o valores. Idealmente, todas las configuraciones consistentes con el modelo deberían ser posibles. A veces, sin embargo, no es posible representar todas las restricciones dentro de un modelo. Un modelo es también una descripción de la estructura genérica y del significado de un sistema. Las descripciones son su *objetivo*, o significado. Un modelo es siempre una abstracción a un cierto nivel. Captura los aspectos *esenciales* de un sistema y omite algunos de los detalles. En los modelos hay que considerar los siguientes aspectos:

Abstracción frente a detalle. Un modelo captura los aspectos esenciales de un sistema y omite otros. Cuáles son esenciales es una cuestión de juicio que depende del propósito del modelo. Esto no es una dicotomía; puede haber un espectro de modelos de precisión creciente. Un lenguaje de modelado no es un lenguaje de programación.

Un lenguaje de modelado puede ser menos exacto a propósito, porque el detalle adicional es irrelevante para el propósito actual. A lo largo de la vida de un proyecto, se pueden utilizar modelos de diferentes niveles de precisión. Un modelo pensado para la generación del código requiere que se traten, por lo menos, algunos elementos del lenguaje de programación. Típicamente, los modelos tienen poca precisión durante el análisis. Ganan en detalle a medida que progresa el ciclo de desarrollo, por lo que los modelos finales, tienen un detalle y precisión considerables.

Especificación frente a implementación. Un modelo puede decir qué hace algo (*especificación*), y también cómo se logra la función (*implementación*). Estos aspectos deben ser separados en el modelado. Es importante determinar el *qué*, antes de invertir muchas horas en el *cómo*. Abstraerse de la implementación es una faceta importante del modelado. Puede haber una cadena de varias relaciones especificación-implementación, en las cuales cada implementación define las especificaciones para la capa siguiente.

Descripción frente a instancia. Los modelos son sobre todo descripción. Las cosas que describen son las instancias, que generalmente aparecen en los modelos sólo como ejemplos. La mayoría de las instancias existen solamente como parte de la ejecución. Algunas veces sin embargo, las instancias de tiempo de ejecución son a veces en sí mismas descripciones de otras cosas. Llamamos a estos objetos híbridos *metadatos*. Es poco realista insistir en que todo es una instancia o una descripción: algo es una instancia o una descripción solamente en referencia a algo más, y la mayoría de las cosas se pueden considerar desde múltiples puntos de vista.

Variaciones en la interpretación. Hay muchas interpretaciones posibles de modelos en un lenguaje de modelado. Uno puede definir ciertos *puntos de variación semántica* —lugares en los cuales son posibles diversas interpretaciones— y asignar a cada interpretación un nombre como *variación semántica*, de modo que uno pueda indicar qué variación está utilizando. Por ejemplo, los usuarios de Smalltalk pueden desear evitar herencia múltiple en un modelo de implementación porque no la tiene el lenguaje de programación. Los usuarios de otros lenguajes de programación lo necesitarían. Los puntos semánticos de variación permiten contemplar diversos modelos de ejecución.

Parte 2: Conceptos de UML



Esta parte contiene una descripción de los conceptos de UML para mostrar cómo encajan en el modelado de un sistema. No pretende describir los conceptos en completo detalle. Para consultar los detalles completos sobre un concepto de UML, véase la sección de la enciclopedia de este libro.

Este capítulo presenta brevemente los conceptos y los diagramas de UML, usando un ejemplo simple. El propósito del capítulo es organizar los conceptos de alto nivel de UML, en un pequeño conjunto de vistas y de diagramas que presentan los conceptos visualmente. Muestra cómo se utilizan los diferentes conceptos describir un sistema, y cómo encajan las vistas unas con otras. Este resumen no pretende ser completo; se omiten muchos conceptos. Para más detalles, véanse los capítulos siguientes, que desarrollan las vistas semánticas de UML, así como el material de referencia detallado en el capítulo de la enciclopedia.

El ejemplo es una taquilla de teatro que ha automatizado sus operaciones. Es un ejemplo inventado con el propósito es destacar varias construcciones de UML en un espacio breve. Está simplificado y deliberadamente no se presenta por completo. La presentación de un modelo completo, de un sistema implementado, ni cabría en un espacio tan pequeño ni destacaría una gama suficiente de construcciones, sin excesiva repetición.

Vistas de UML

No hay ninguna línea entre los diferentes conceptos y las construcciones en UML, pero, por conveniencia, nosotros los dividimos en varias vistas. Una vista es simplemente un subconjunto de UML que modela construcciones que representan un aspecto de un sistema. La división en diversas vistas es algo arbitraria, pero esperamos que sea intuitiva. Una o dos clases de diagramas proporcionan una notación visual para los conceptos de cada vista.

En el nivel superior, las vistas se pueden dividir en tres áreas: clasificación estructural, comportamiento dinámico, y gestión del modelo.

La clasificación estructural describe los elementos del sistema y sus relaciones con otros elementos. Los clasificadores incluyen clases, casos del uso, componentes, y nodos y elementos proporcionan la base sobre la cual se construye el comportamiento dinámico. La clasificación de las vistas incluye la vista estática, la vista de casos de uso, y la vista de implementación.

El comportamiento dinámico describe el comportamiento de un sistema en el tiempo. El comportamiento se puede describir como serie de cambios a las fotos del sistema dibujadas a partir de la visión estática. Las vistas de comportamiento dinámico incluyen vista de la máquina de estados, la vista de actividad, y la vista de interacción.

La gestión del modelo describe la organización de los propios modelos en unidades jerárquicas. El paquete es la unidad genérica de organización para los modelos. Los paquetes especiales incluyen a los modelos y a los subsistemas. La vista de gestión del modelo cruza las otras vistas y las organiza para el trabajo de desarrollo y el control de configuración.

Tabla 3.1 Vistas y diagramas de UML

Área	Vista	Diagramas	Conceptos Principales
estructural	vista estática	diagrama de clases	clase, asociación, generalización, dependencia, realización, interfaz
	vista de casos de uso	diagrama de casos de uso	caso de uso, actor, asociación, extensión, inclusión, generalización de casos de uso
	vista de implementación	diagrama de componentes	componente, interfaz, dependencia, realización
	vista de despliegue	diagrama de despliegue	nodo, componente, dependencia, localización
dinámica	vista de máquina de estados	diagrama de estados	estado, evento, transición, acción
	vista de actividad	diagrama de actividad	estado, actividad, transición de terminación, división, unión
	vista de interacción	diagrama de secuencia	interacción, objeto, mensaje, activación
		diagrama de colaboración	colaboración, interacción, rol de colaboración, mensaje
gestión del modelo	vista de gestión del modelo	diagrama de clases	paquete, subsistema, modelo
extensión de UML	todas	todos	restricción, estereotipo, valores etiquetados

UML también contiene varias construcciones previstas para proporcionar una capacidad limitada, pero útil, de extensión. Estas construcciones incluyen restricciones, estereotipos, y valores etiquetados y son aplicables a los elementos de todas las vistas.

La Tabla 3.1 muestra las vistas de UML y los diagramas que las muestran, así como los principales conceptos relevantes de cada vista. Esta tabla no se debe tomar como un rígido sistema de reglas sino simplemente como guía para el uso normal, dado que se permite la mezcla de vistas.

Vista estática

La vista estática modela los conceptos del dominio de la aplicación, así como los conceptos internos inventados como parte de la implementación de la aplicación. Esta visión es estática porque no describe el comportamiento del sistema dependiente del tiempo, que se describe en otras vistas. Los componentes principales de la vista estática son las clases y sus relaciones: asociación, generalización, y varias clases de dependencia, tales como realización y uso. Una clase es la descripción de un concepto del dominio de la aplicación o de la solución de la apli-

cación. Las clases son el centro alrededor del cual se organiza la vista de clases; otros elementos pertenecen o se unen a las clases. La visión estática se exhibe en los diagramas de clases, llamados así porque su objetivo principal es la descripción de clases.

Las clases se dibujan como rectángulos. Las listas de atributos y de operaciones se muestran en compartimentos separados. Los compartimentos pueden ser suprimidos cuando no es necesario el detalle completo. Una clase puede aparecer en varios diagramas. Sus atributos y operaciones se suprimen, a menudo en todos menos en un diagrama.

Las relaciones entre clases se dibujan como las líneas que conectan rectángulos de clases. Las diversas clases de relaciones se diferencian por la textura de la línea y por los adornos en las líneas o en sus extremos.

La Figura 3.1 muestra un diagrama de clases de la aplicación de la taquilla. Este diagrama contiene parte del modelo del dominio “venta de entradas”. Muestra varias clases importantes, tales como **Cliente**, **Reserva**, **Entrada**, y **Representación**. Los clientes pueden tener muchas reservas, pero cada reserva es hecha por un cliente. Las reservas son de dos clases: suscripción y reservas individuales. Ambos reservan entradas: en un caso, solamente una entrada; en el otro caso, varias entradas. Cada entrada es parte de una suscripción o de una reserva individual, pero

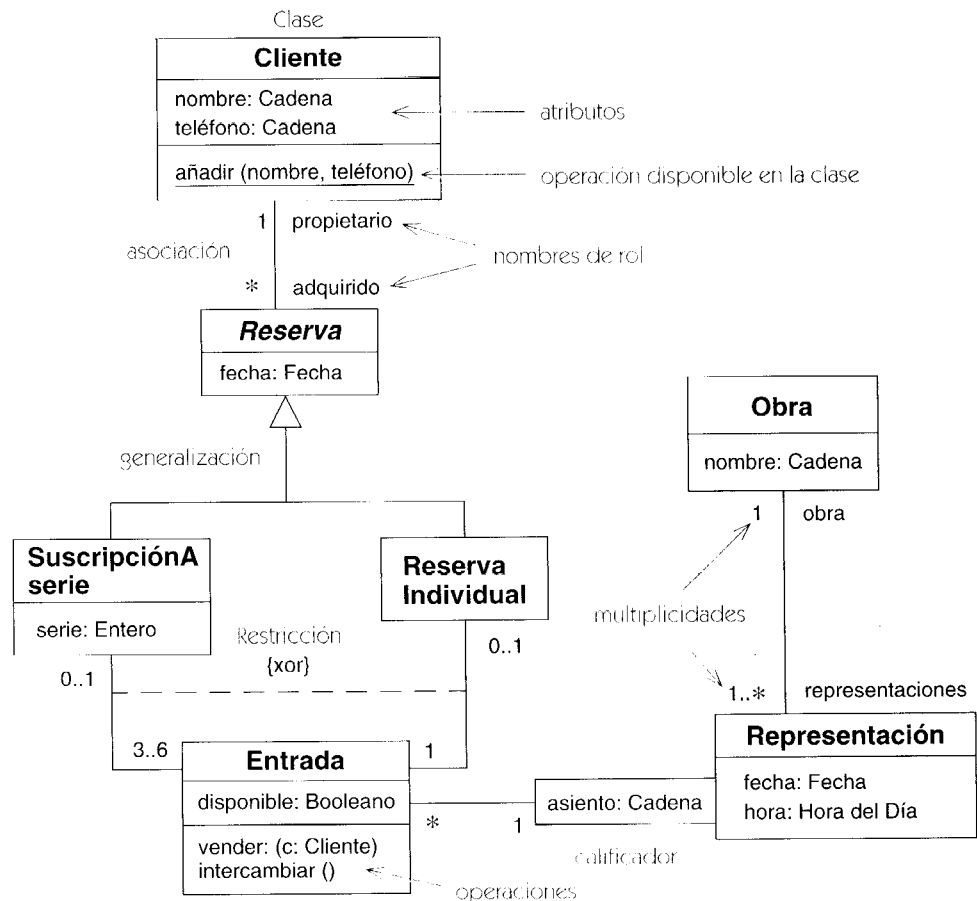


Figura 3.1 Diagrama de clases

no de ambas. Cada representación tiene muchas entradas disponibles, cada una con un número de asiento único. Una representación se puede identificar por una obra, una fecha, y una hora.

Las clases se pueden describir con varios niveles de precisión y concreción. Al empezar el diseño, el modelo captura los aspectos más lógicos del problema. En las fases posteriores, el modelo también capta decisiones de diseño y detalles de la implementación. La mayoría de las vistas tienen un comportamiento evolutivo similar.

Vista de los casos de uso

La vista de los casos de uso modela la funcionalidad del sistema según lo perciben los usuarios externos, llamados actores. Un caso de uso es una unidad coherente de funcionalidad, expresada como transacción entre los actores y el sistema. El propósito de la vista de casos de uso es enumerar a los actores y los casos de uso, y demostrar qué actores participan en cada caso de uso.

La Figura 3.2 muestra un diagrama de casos de uso para el ejemplo de la taquilla. Los actores son el vendedor, el supervisor, y el quiosco. El quiosco es otro sistema que acepta pedidos de un cliente. El cliente no es actor en la aplicación de la taquilla, porque no está conectado directamente con la aplicación. Los casos de uso incluyen el comprar entradas, a través del quiosco o del vendedor, comprar suscripciones (solamente a través del vendedor), y examinar las ventas totales (a petición del supervisor).

El comprar entradas y el comprar las suscripciones incluyen un fragmento común —que es hacer cargos al servicio de la tarjeta de crédito—. (La descripción completa de la taquilla im-

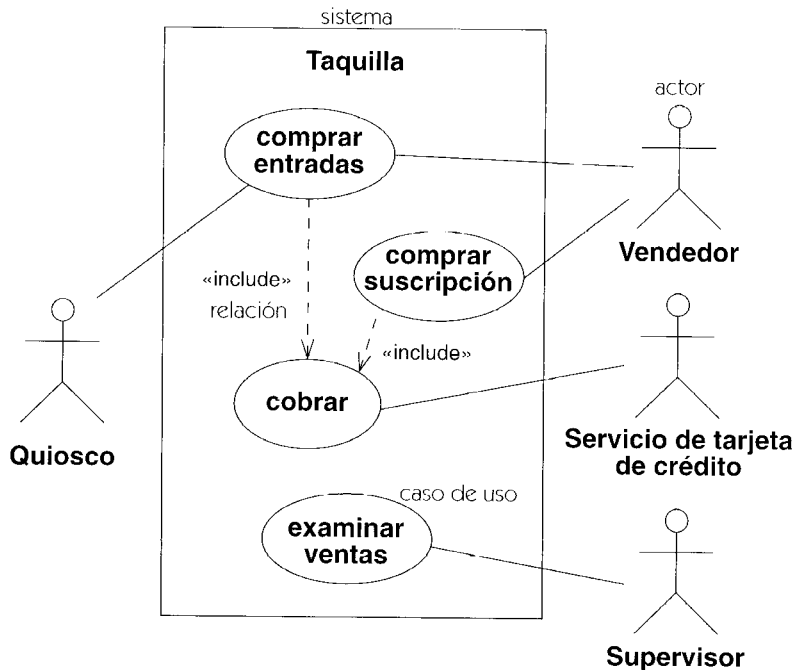


Figura 3.2 Diagrama de casos de uso

plicaría muchos otros casos de uso, tales como cambio de entradas y comprobación de disponibilidad.)

Los casos de uso se pueden también describir en varios niveles de detalle. Se pueden sacar partes como factor común y ser descritos en términos de otros casos de uso más simples. Un caso del uso se implementa como una colaboración en la vista de interacción.

Vista de interacción

La vista de interacción describe secuencias de intercambios de mensajes entre los roles que implementan el comportamiento de un sistema. Un rol de clasificador, o simplemente “rol”, es la descripción de un objeto, que desempeña un determinado papel dentro de una interacción, distinto de los otros objetos de la misma clase. Esta visión proporciona una vista integral del comportamiento de un sistema —es decir, muestra el flujo de control a través de muchos objetos—. La vista de interacción se exhibe en dos diagramas centrados en distintos aspectos: diagramas de secuencia y diagramas de colaboración.

El diagrama de secuencia

Un diagrama de secuencia muestra un conjunto de mensajes, dispuestos en una secuencia temporal. Cada rol en la secuencia se muestra como una línea de vida, es decir, una línea vertical

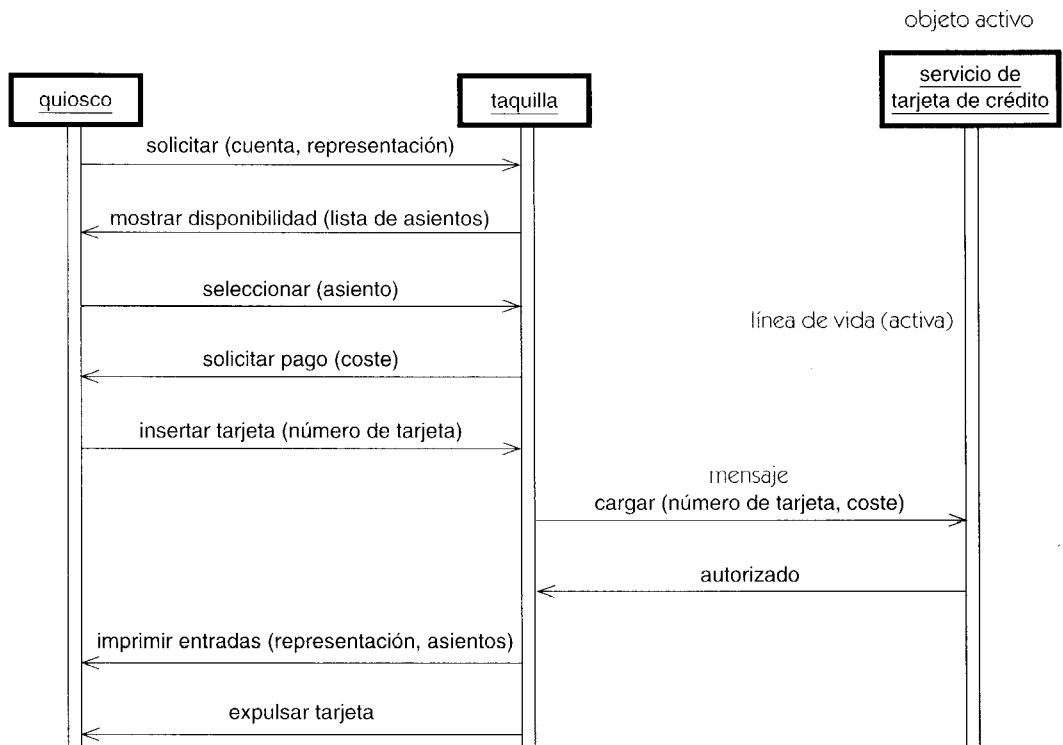


Figura 3.3 Diagrama de secuencia

que representa el rol durante cierto plazo de tiempo, con la interacción completa. Los mensajes se muestran como flechas entre las líneas de vida. Un diagrama de secuencia puede mostrar un escenario, es decir, una historia individual de una transacción.

Un uso de un diagrama de secuencia es mostrar la secuencia del comportamiento de un caso del uso. Cuando está implementado el comportamiento, cada mensaje en un diagrama de secuencia corresponde a una operación en una clase, a un evento disparador, o a una transición en una máquina de estados.

La Figura 3.3 muestra un diagrama de secuencia para el caso de uso **comprar entrada**. Este caso de uso lo inicia el cliente en el quiosco, comunicándose con la taquilla. Los pasos para el caso de uso **hacer cargos** se incluyen en la secuencia, que implica la comunicación con el quiosco y el servicio de la tarjeta de crédito. Este diagrama de secuencia está en una primera etapa de desarrollo y no muestra los detalles completos de la interfaz de usuario. Por ejemplo, la forma exacta de la lista de asientos y el mecanismo de especificar asientos, deben ser definidos aún, pero la comunicación esencial de la interacción ha sido especificada por el caso de uso.

El diagrama de colaboración

Una colaboración modela los objetos y los enlaces significativos dentro de una interacción. Los objetos y los enlaces son significativos solamente en el contexto proporcionado por la interacción. Un rol describe un objeto, y un rol en la asociación describe un enlace dentro de una colaboración. Un diagrama de colaboración muestra los roles en la interacción en una disposición geométrica (Figura 3.4). Los mensajes se muestran como flechas, ligadas a las líneas de la relación, que conectan a los roles. La secuencia de mensajes, se indica con los números secuenciales que preceden a las descripciones del mensaje.

Un uso de un diagrama de colaboración es mostrar la implementación de una operación. La colaboración muestra los parámetros y las variables locales de la operación, así como asociaciones más permanentes. Cuando se implementa el comportamiento, la secuencia de los mensajes corresponde a la estructura de llamadas anidadas y el paso de señales del programa.

La Figura 3.4 muestra un diagrama de colaboración para la interacción de la reserva de entradas en una fase ulterior del desarrollo. La colaboración muestra la interacción entre objetos internos en la aplicación para reservar entradas. La petición llega del quiosco y se utiliza para encontrar la base de datos de la representación deseada en el conjunto de todas las representaciones. El puntero **db** que es devuelto al objeto **vendedor de entradas** representa un enlace local transitorio a una base de datos de representaciones, que persiste durante la interacción y después se desecha. El vendedor de entradas solicita un número de asientos para la representación; se encuentra una selección de asientos en varias gamas de precios, se bloquea temporalmente, y se devuelve al quiosco para la selección de los clientes. Cuando el cliente hace una selección de la lista de asientos, se obtienen los asientos seleccionados y el resto son liberados.

Tanto los diagramas de secuencia como los diagramas de colaboración muestran interacciones, pero acentúan aspectos diferentes. Un diagrama de secuencia muestra secuencias en el tiempo como dimensión geométrica, pero las relaciones entre roles son implícitas. Un diagrama de colaboración muestra las relaciones entre roles geoméricamente, y relaciona los mensajes con las relaciones, pero las secuencias temporales están menos claras, porque vienen dadas por los números de secuencia. Cada diagrama debe ser utilizado cuando su aspecto principal es el foco de atención.

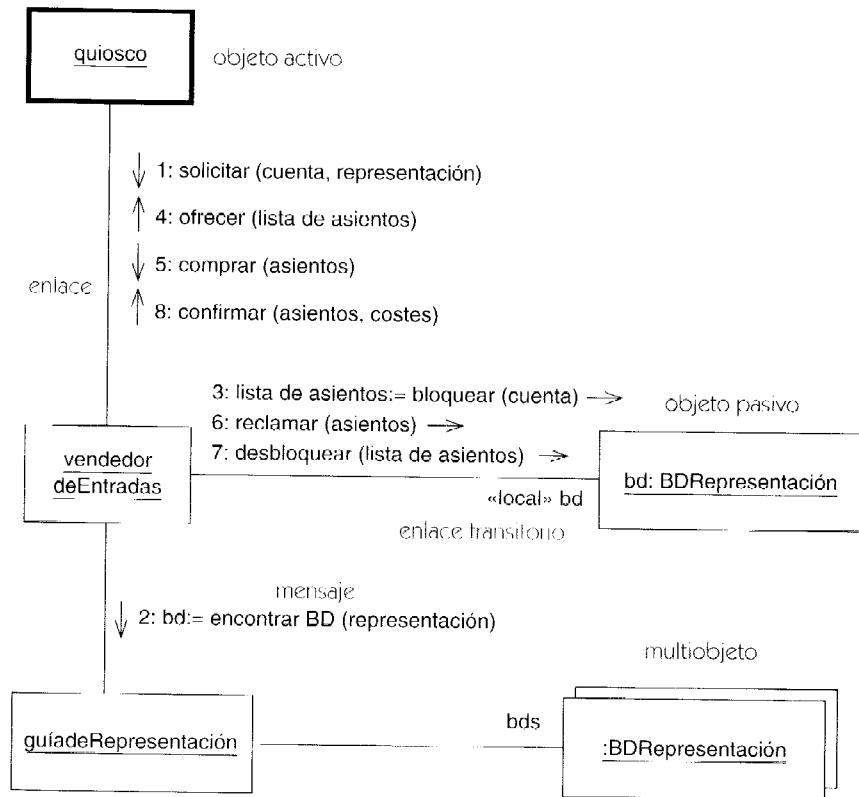


Figura 3.4 Diagrama de colaboración

Vista de la máquina de estados

Una máquina de estados modela las posibles historias de vida de un objeto de una clase. Una máquina de estados contiene los estados conectados por transiciones. Cada estado modela un período de tiempo, durante la vida de un objeto, en el que satisface ciertas condiciones. Cuando ocurre un evento, se puede desencadenar una transición que lleve el objeto a un nuevo estado. Cuando se dispara una transición, se puede ejecutar una acción unida a la transición. Las máquinas de estados se muestran como diagramas de estados.

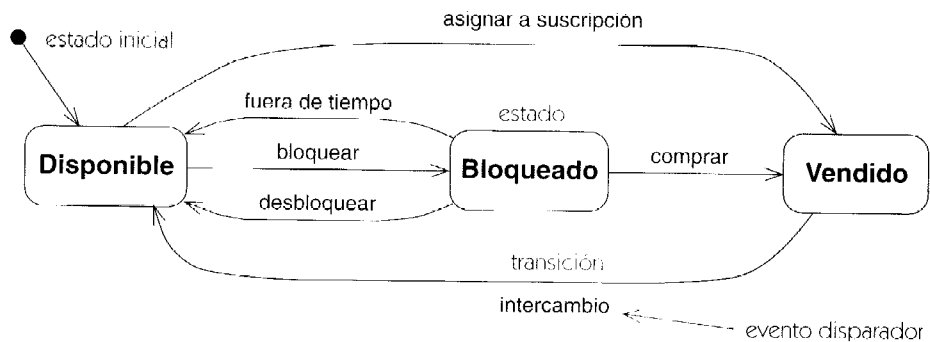


Figura 3.5 Diagrama de estados

La Figura 3.5 muestra un diagrama de estados de la historia de una entrada para una representación. El estado inicial de una entrada (representado por un punto negro) es el estado **disponible**. Antes del comienzo de la temporada, se asignan los asientos para los suscriptores de la temporada. Las entradas individuales compradas interactivamente primero se bloquean mientras el cliente hace una selección. Después de esto, se venden o se liberan si no se eligen. Si el cliente tarda demasiado en hacer una selección, finaliza el tiempo de la transacción y se libera el asiento. Los asientos vendidos para suscriptores de temporada, se pueden cambiar para otras representaciones, en cuyo caso, vuelven a estar disponibles otra vez.

Las máquinas de estados se pueden utilizar para describir interfaces de usuario, controladores de dispositivo, y otros subsistemas reactivos. También pueden usarse para describir los objetos pasivos que pasan por varias fases cualitativas distintas, durante su tiempo de vida, cada una de las cuales tiene su propio comportamiento especial.

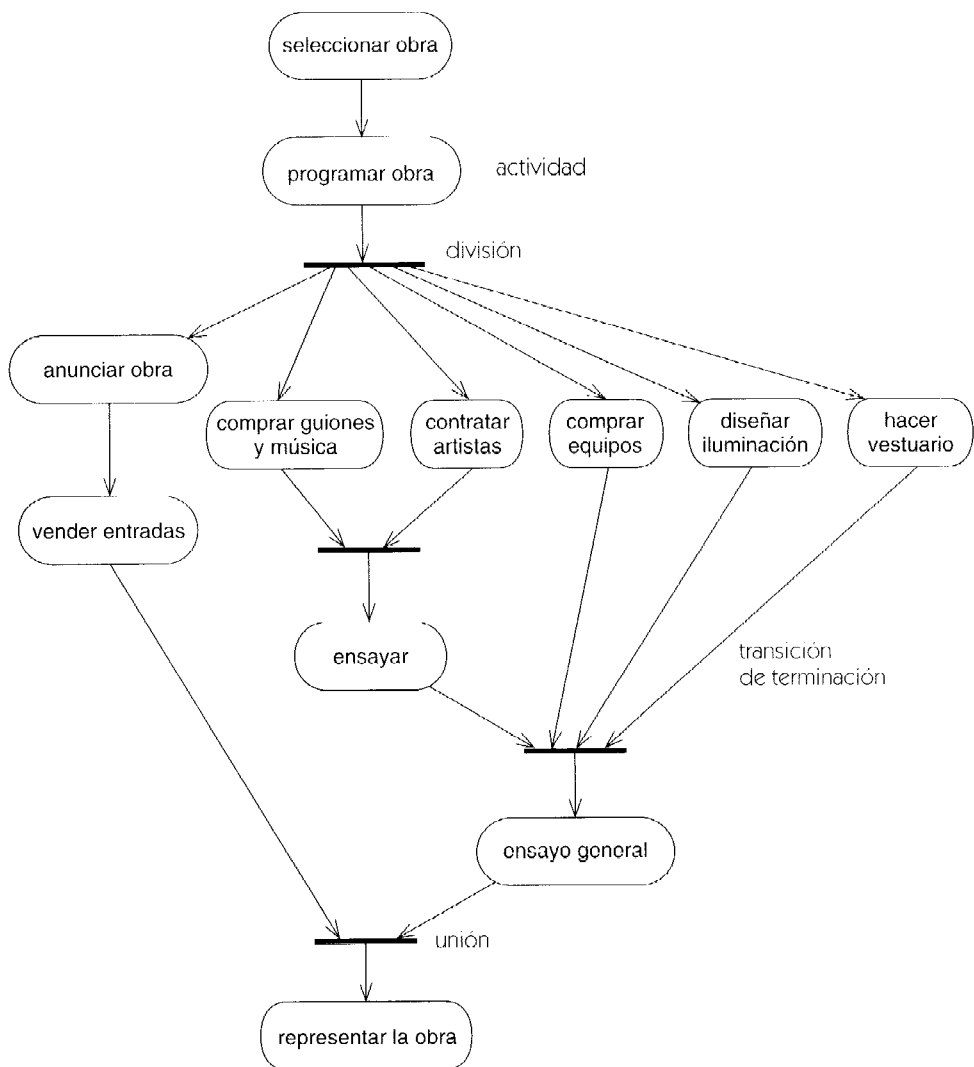


Figura 3.6 Diagrama de actividades

Vista de actividades

Un grafo de actividades es una variante de una máquina de estados, que muestra las actividades de computación implicadas en la ejecución de un cálculo. Un estado de actividad representa una actividad: un paso en el flujo de trabajo o la ejecución de una operación. Un grafo de actividades describe grupos secuenciales y concurrentes de actividades. Los grafo de actividades se muestran en diagramas de actividades.

La Figura 3.6 muestra un diagrama de actividades para la taquilla. Este diagrama muestra las actividades implicadas en montar una obra. (¡No tome este ejemplo demasiado en serio si tiene experiencia teatral!) Las flechas muestran dependencias secuenciales por ejemplo, las representaciones deben ser seleccionadas antes de poder planificarlas. Las barras horizontales representan bifurcaciones o uniones de control. Por ejemplo, después de planificar la obra, el teatro puede comenzar a darle publicidad, a comprar guiones, a contratar artistas, a construir decorados, a diseñar la iluminación, y a hacer los trajes, todo concurrentemente. Antes de que el ensayo pueda comenzar, sin embargo, se deben haber encargado los guiones y contratado a los artistas.

Este ejemplo muestra un diagrama de actividades, cuyo propósito es modelar los procesos reales de una organización humana. El modelado de tales negocios es un propósito importante de los diagramas de actividades, pero los diagramas de actividades se pueden también utilizar para modelar actividades software. Un diagrama de actividades es provechoso para entender el comportamiento de alto nivel de la ejecución de un sistema, sin profundizar en los detalles internos de los mensajes, lo que requeriría un diagrama de colaboración.

Los parámetros de entrada y de salida de una acción se pueden mostrar usando las relaciones de flujo que conectan la acción y un estado de flujo del objeto.

Vistas físicas

Las vistas anteriores modelan los conceptos de la aplicación desde un punto de vista lógico. Las vistas físicas modelan la estructura de la implementación de la aplicación por sí misma, su organización en componentes, y su despliegue en nodos ejecución. Estas vistas proporcionan una oportunidad de establecer correspondencias entre las clases y los componentes de implementación y nodos.

Hay dos vistas físicas: la vista de implementación y la vista de despliegue.

La vista de implementación modela los componentes de un sistema —a partir de los cuales se construye la aplicación— así como las dependencias entre los componentes, para poder determinar el impacto de un cambio propuesto. También modela la asignación de clases y de otros elementos del modelo a los componentes.

La vista de implementación se representa en diagramas componentes. La Figura 3.7 muestra un diagrama de componentes para el sistema de la taquilla.

Hay tres interfaces de usuario: la de los clientes que usan un quiosco, la de los vendedores que usan el sistema de reserva automatizado, y la de los supervisores que hacen consultas sobre las ventas de entradas. Hay un componente vendedor de entradas que ordena las peticiones de los quioscos y de los vendedores; un componente que procesa los cargos a la tarjeta de crédito; y la base de datos que contiene la información de la entrada. El diagrama de componentes mues-

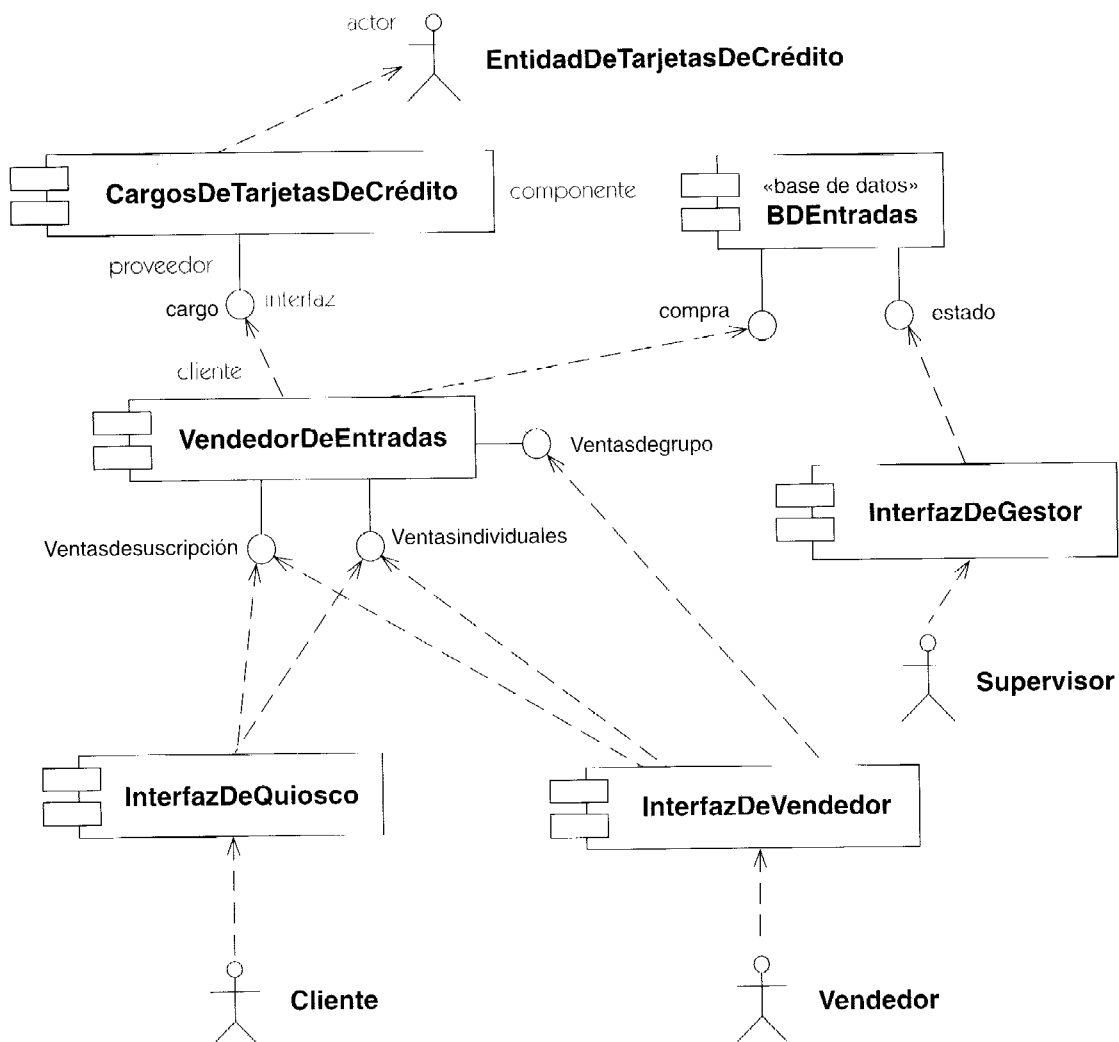


Figura 3.7 Diagrama de componentes

tra los tipos de componentes del sistema; una configuración particular de la aplicación puede tener más de una copia de un componente.

Un círculo pequeño con un nombre es una interfaz —un conjunto coherente de servicios—. Una línea sólida que va desde un componente a una interfaz, indica que el componente proporciona los servicios de la interfaz.

Una flecha de guiones de un componente a una interfaz indica que el componente requiere los servicios proporcionados por interfaz. Por ejemplo, las ventas de suscripciones y las ventas de grupos de entradas, son proporcionadas por el componente “vendedor de entradas”; las ventas de suscripciones son accesibles tanto para los quioscos como para los vendedores, pero las ventas de grupos sólo son accesibles para un vendedor.

La vista de despliegue representa la disposición de las instancias de componentes de ejecución en instancias de nodos. Un nodo es un recurso de ejecución, tal como una computadora, un

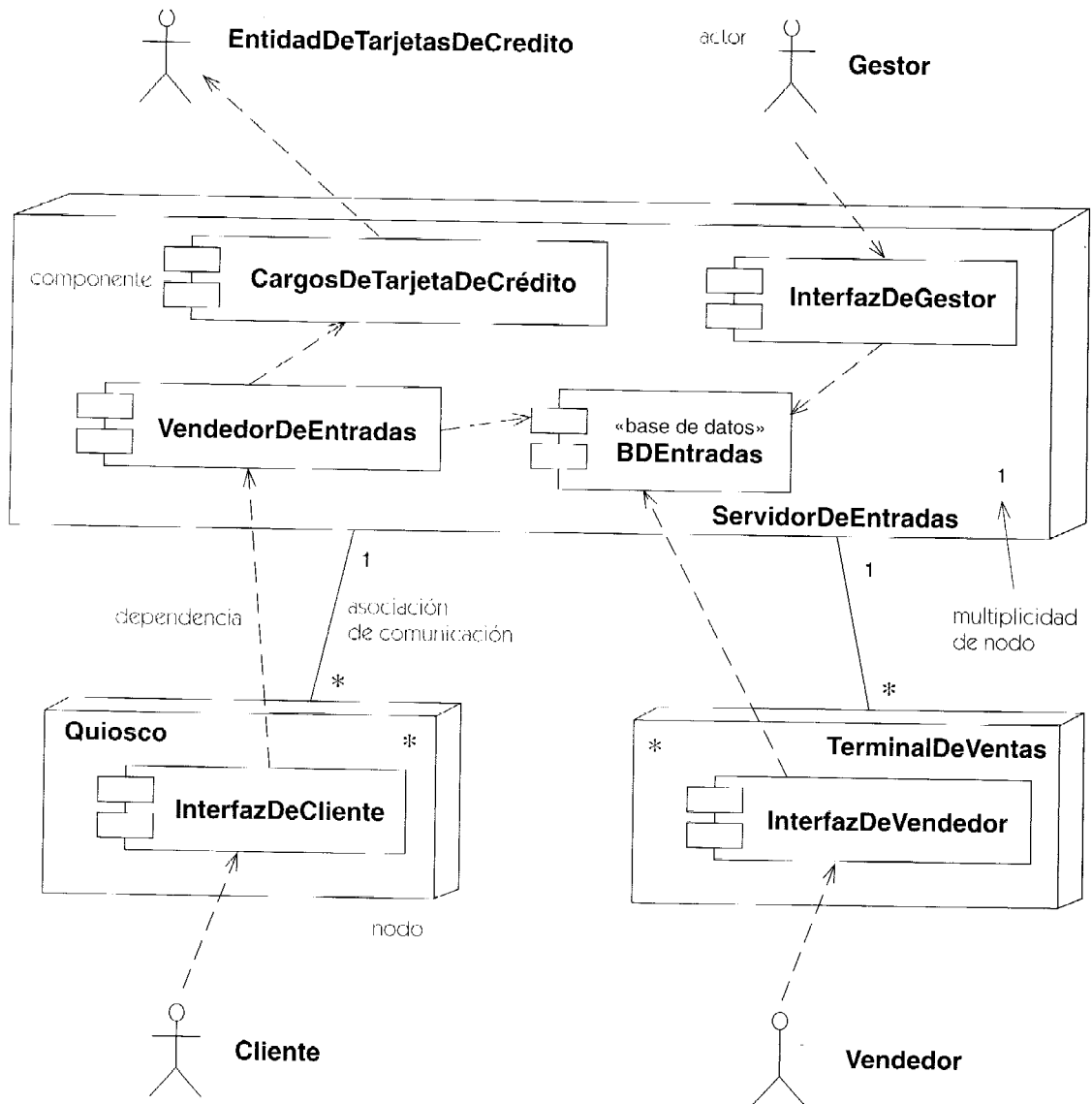


Figura 3.8 Diagrama de despliegue (nivel de descriptor)

dispositivo, o memoria. Esta vista permite determinar las consecuencias de la distribución y de la asignación de recursos.

La vista de despliegue se representa en diagramas de despliegue. La Figura 3.8 muestra un diagrama de despliegue del nivel de descriptor para el sistema de taquilla. Este diagrama muestra los tipos de nodos del sistema y los tipos de componentes que contienen. Un nodo se representa como un cubo.

La Figura 3.9 muestra un diagrama de despliegue del nivel de instancia, para el sistema de taquilla. El diagrama muestra los nodos individuales y sus enlaces, en una versión particular del sistema. La información de este modelo es consistente con la información del nivel de descriptor de la Figura 3.8.

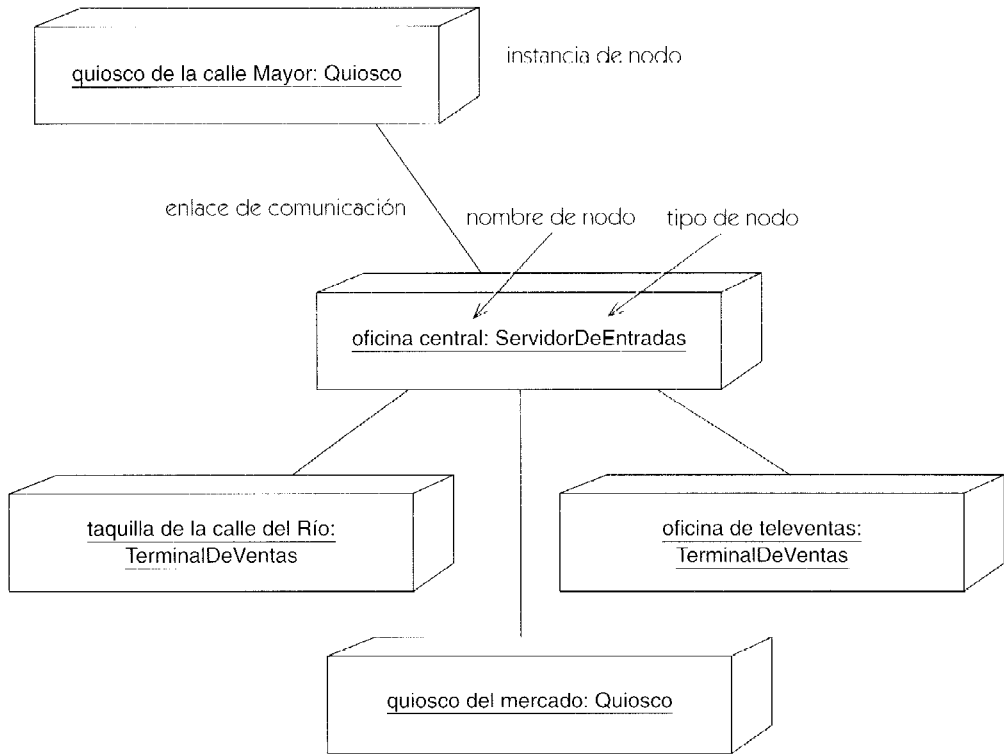


Figura 3.9 Diagrama de despliegue (nivel de instancia)

Vista de gestión del modelo

La vista de gestión del modelo modela la organización del modelo en sí mismo. Un modelo abarca un conjunto de paquetes que contienen los elementos del modelo, tales como clases, máquinas de estados, y casos de uso. Los paquetes pueden contener otros paquetes: por lo tanto, un modelo señala un paquete raíz, que contiene indirectamente todo el contenido del modelo. Los paquetes son unidades para manipular el contenido de un modelo, así como unidades para el control de acceso y el control de configuración. Cada elemento del modelo pertenece a un paquete o a otro elemento.

Un modelo es una descripción completa de un sistema, con una determinada precisión, desde un punto de vista. Puede haber varios modelos de un sistema desde distintos puntos de vista; por ejemplo, un modelo de análisis y un modelo de diseño. Un modelo se representa como una clase especial de paquete.

Un subsistema es otro paquete especial. Representa una porción de un sistema, con una interfaz perfectamente determinada, que puede ser implementado como un componente distinto.

Generalmente, la información de gestión del modelo se representa en diagramas de clases.

La Figura 3.10 muestra la descomposición de la totalidad del sistema de teatro en paquetes y sus relaciones de dependencia. El subsistema de taquilla incluye los ejemplos anteriores de este capítulo; el sistema completo también incluye operaciones del teatro y subsistemas de planificación. Cada subsistema consta de varios paquetes.

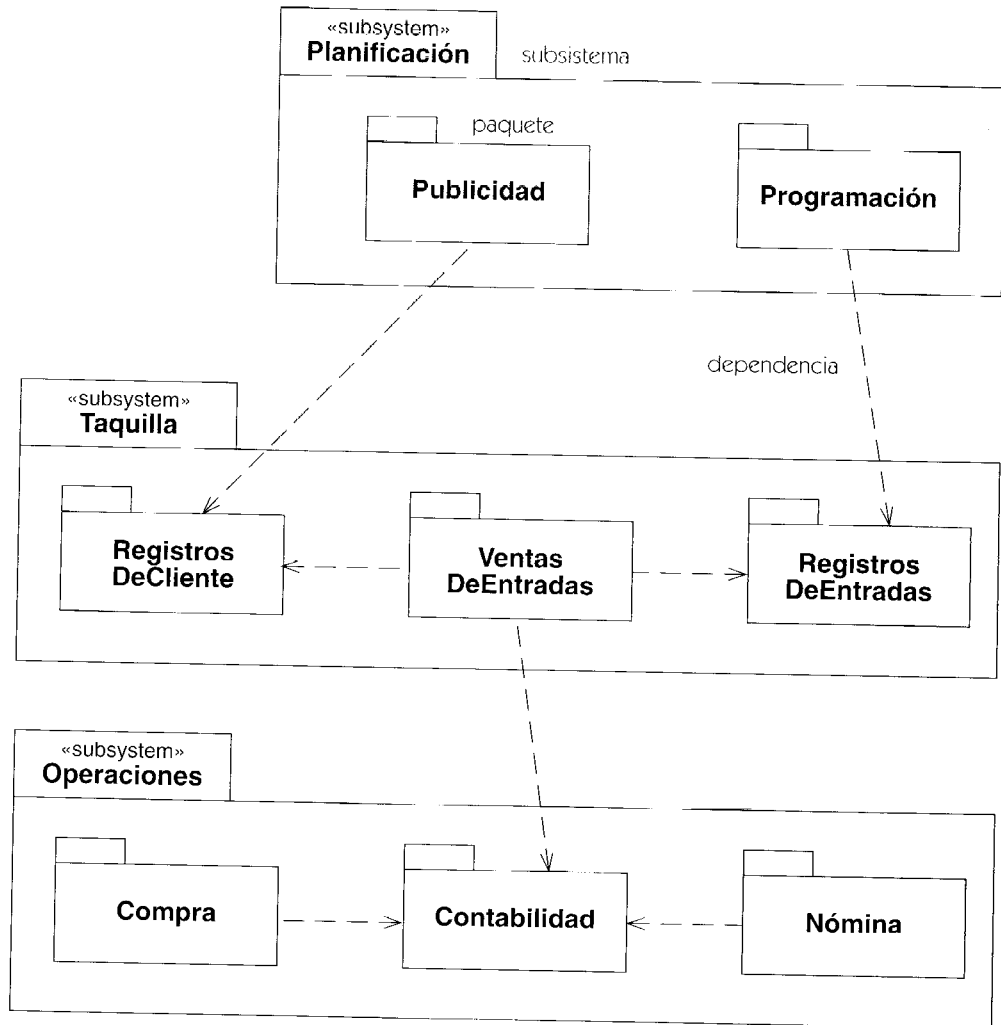


Figura 3.10 Paquetes

Construcciones de extensión

UML incluye tres construcciones principales de extensión: restricciones, estereotipos, y valores etiquetados. Una restricción es una declaración textual de una relación semántica expresada en un cierto lenguaje formal o en lenguaje natural. Un estereotipo es una nueva clase de elemento del modelo, ideada por el modelador, y basada en un tipo existente de elemento del modelo. Un valor etiquetado es una porción de información con nombre, unida a cualquier elemento del modelo.

Estas construcciones permiten muchas clases de extensiones a UML, sin requerir cambios al metamodelo básico de UML. Pueden ser utilizadas para crear versiones adaptadas a un área de aplicación.

La Figura 3.11 muestra ejemplos de restricciones, estereotipos, y de valores etiquetados. La restricción en la clase **Obra** asegura que los nombres de las obras sean únicos. La Figura 3.1

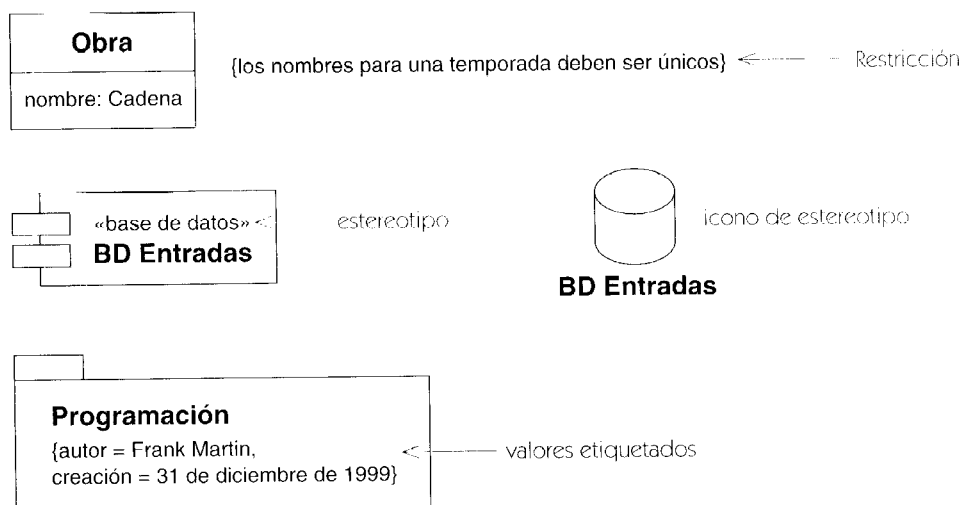


Figura 3.11 Construcciones de extensión

muestra una restricción **xor** entre dos asociaciones; un objeto sólo puede tener un enlace desde una de ellas a la vez. Las restricciones son útiles para representar sentencias que se pueden expresar en un lenguaje textual pero que no tienen soporte directo mediante construcciones de UML.

El estereotipo en el componente **DBEntrada** indica que el componente es una base de datos, lo que permite omitir qué interfaces soporta, puesto que son las soportadas por todas las bases de datos. Los modeladores pueden agregar nuevos estereotipos para representar elementos especiales. Un conjunto de restricciones implicadas, valores etiquetados, o características de la generación del código, pueden ser asociados a un estereotipo. Un modelador puede definir un icono para un nombre de estereotipo, como ayuda visual, según lo mostrado en el diagrama. No obstante, la forma textual se puede utilizar siempre.

Los valores etiquetados en el paquete **Programación** muestran que Frank Martín es responsable de acabarla antes del final del milenio. Se puede unir a un elemento del modelo cualquier pieza de información arbitraria, como un valor etiquetado bajo un nombre elegido por el modelador. Los valores de texto son especialmente útiles para la información de gestión de proyecto y para los parámetros de generación del código. La mayoría de los valores etiquetados serían almacenados, como información emergente, dentro de una herramienta de edición, y no aparecerán generalmente en las figuras impresas.

Conexiones entre vistas

Dentro de un modelo coexisten distintas vistas y sus elementos tienen muchas conexiones, algunas de las cuales se muestran en la Tabla 3.2. Esta tabla no pretende ser completa, sino mostrar algunas de las relaciones principales entre elementos de diversas vistas.

Tabla 3.2 Algunas Relaciones entre Elementos de Diferentes Vistas

<i>Elemento</i>	<i>Elemento</i>	<i>Relación</i>
clase	máquina de estados	propiedad
operación	interacción	realización
caso de uso	colaboración	realización
caso de uso	instancia de interacción	escenario de ejemplo
instancia de un componente	instancia de un nodo	localización
acción	operación	llamada
acción	señal	envío
actividad	operación	llamada
mensaje	acción	invocación
paquete	clase	propiedad
rol	clase	clasificación

